TU Clausthal

Master's Thesis

by

Yu Zhang

Explain Reality: Interactive Explanations with Foundation Models on the Meta Quest 3

1st Supervisor: **Prof. Dr. Christian Bartelt**

2nd Supervisor: **Prof. Dr. Rüdiger Ehlers**

10th June 2026

Institute for Software and Systems Engineering
Clausthal University of Technology

Declaration of Authorship

I have read and understood the guidelines of the Technical University of Clausthal and those published in the ISSE bulletin, including those regarding the use of literature and other sources. I confirm that I have prepared this thesis independently by myself. Any information taken from other sources and being reproduced in this thesis is clearly referenced.

In terms of the general examination regulations, this work has not yet been submitted to any other examination division.

I hereby agree that my master's thesis may be exhibited in the institute's library and kept for inspection.

Clausthal-Zellerfeld, 10th June 2026	
Location, Date	Yu Zhang

Abstract

Explain Reality studies how foundation models can give useful interactive explanations inside an immersive simulator without becoming unbounded conversational agents. The motivating setting is a Meta Quest 3 training task in Digital Combat Simulator World: learners perform the F/A-18C Lot 20 cold-start procedure while the tutor must explain the next action from the aircraft state that is actually present. In this setting, "reality" is not the physical aircraft but a live, high-load, multimodal simulation state. An explanation is useful only if it can be traced to inspectable evidence.

This thesis presents SimTutor, an evidence-grounded procedural tutoring runtime that combines procedure-pack rules, DCS-BIOS telemetry, retrieved procedure knowledge, structured response validation, replayable logs, and a vision-language model (VLM) layer for cockpit display facts. The main machine learning contribution is a small-data LoRA adaptation pipeline for structured visual fact extraction. A first 8-fact ontology revealed that compound targets such as procedural completion states were prone to high-risk false positives. The revised 13-fact ontology instead asks the VLM for local visual evidence atoms and leaves procedural interpretation to deterministic downstream rules.

On two independent 13-fact holdouts with verified zero exact overlap to the training set, the Qwen3.5 LoRA adapter trained with the Run-003 + Run-005x2 recipe is the strongest offline line. On Run-002 newfacts, fact accuracy rises from 0.8677 to 0.9908 and sample exact match from 0.10 to 0.88; on Run-004 random, fact accuracy rises from 0.8038 to 0.9931 and sample exact match from 0.03 to 0.92. Critical false positives fall from 11 to 4 on Run-002 and from 70 to 8 on Run-004. A matched Qwen3.6-27B experiment shows a stronger base model and a deployable LoRA candidate, but it does not surpass the Qwen3.5 LoRA line on the current holdouts. Gemma improves substantially under the same data recipe and is more conservative on some critical facts, but remains below Qwen3.5 in overall exact-match behaviour.

A formative non-random Quest 3 pilot with nine participants complements the technical evidence. The participant pool included novice, intermediate, and expert self-

reported profiles. Four participants used the tutor and five performed a passive without-tutor condition. All four tutor-condition participants completed the manually reviewed 33-step procedure, whereas none of the five without-tutor participants did. The tutor condition showed zero manual omission errors but introduced assistance-coupled commission and prompting errors in several trials. Raw NASA-TLX and usability/trust responses are reported descriptively only; P02's questionnaire is excluded because of a metadata mismatch, and P08 uses manual completion coding. The pilot therefore supports feasibility, instrumentation, and error-pattern claims, not causal claims about learning gains, workload reduction, retention, transfer, or task speed.

The thesis argues that reliable interactive explanation in immersive training is best treated as an evidence-design problem: foundation models should explain the live task state only through evidence that the runtime can observe, validate, and replay.

Zusammenfassung

Explain Reality untersucht, wie Foundation Models in einer immersiven Simulationsumgebung interaktive Erklärungen geben können, ohne als unbegrenzte Dialogsysteme eingesetzt zu werden. Der Anwendungskontext ist ein Training mit der Meta Quest 3 in Digital Combat Simulator World, in dem Teilnehmende den Kaltstart der F/A-18C Lot 20 ausführen. Die Erklärung des Systems muss sich auf den tatsächlich beobachtbaren Simulatorzustand beziehen. "Realität" bedeutet in dieser Arbeit daher den laufenden, multimodalen und kognitiv anspruchsvollen Zustand der Simulation.

Die Arbeit stellt SimTutor vor, eine evidenzbasierte Laufzeitumgebung für prozedurales Tutoring. Sie kombiniert deklarative Prozedurregeln, DCS-BIOS- Telemetrie, Retrieval von Prozedurwissen, strukturierte Antwortvalidierung, Replay-Logs und ein Vision-Language Model für Cockpit-Anzeigen. Der zentrale Machine-Learning-Beitrag ist eine kleine LoRA-Anpassung für strukturierte visuelle Fakten. Eine frühe 8-Fact-Ontologie zeigte, dass zusammengesetzte Zielzustände riskante False Positives erzeugen können. Die revidierte 13-Fact-Ontologie trennt deshalb lokale visuelle Evidenz von der späteren prozeduralen Interpretation.

Auf zwei unabhängigen 13-Fact-Holdouts ist die Qwen3.5-LoRA-Linie mit Run-003 + Run-005x2 die stärkste Offline-Konfiguration. Die Fact Accuracy steigt auf Run-002 newfacts von 0,8677 auf 0,9908 und auf Run-004 random von 0,8038 auf 0,9931; gleichzeitig sinken kritische False Positives von 11 auf 4 beziehungsweise von 70 auf 8. Qwen3.6-27B liefert eine starke und deploybare Kandidatenlinie, übertrifft die Qwen3.5-LoRA-Linie auf den aktuellen Holdouts jedoch nicht. Gemma profitiert ebenfalls von derselben Datenrezeptur, bleibt aber beim exakten Sample-Match hinter Qwen3.5.

Eine formative, nicht randomisierte Quest-3-Pilotstudie mit neun Teilnehmenden ergänzt die technische Evaluation. Die Stichprobe enthielt selbstberichtete Novizen, intermediäre und erfahrene Teilnehmende. Vier Teilnehmende nutzten den Tutor, fünf absolvierten eine passive Bedingung ohne Tutor. Alle vier Tutor-Teilnehmenden vollendeten die manuell geprüfte 33-Schritt-Prozedur; keiner der fünf Teilnehmenden ohne Tutor erreichte den Endzustand. Die Pilotdaten belegen Machbarkeit, Instrumentierung

und Fehlermuster, aber keine kausalen Effekte auf Lernen, Arbeitsbelastung, Retention, Transfer oder Geschwindigkeit.

Acknowledgements

I would like to thank my supervisor for guidance and feedback throughout this thesis. I also thank the pilot study participants for their time, patience, and careful engagement during the VR sessions.

Declaration on the Use of AI Tools

AI-assisted tools, including ChatGPT/Codex and Claude Code, were used during the preparation of this thesis for language editing, restructuring suggestions, LaTeX consistency checks, citation-gap auditing, and inspection of repository artefacts under the author's direction. These tools were not used to fabricate experimental data, participant results, or bibliographic metadata. All claims, numerical results, citations, interpretations, and final wording remain the responsibility of the author.

Contents

List of Figures

List of Tables

1 Introduction

1.1 Motivation: Explaining a Live Training Reality

The title of this thesis, *Explain Reality: Interactive Explanations with Foundation Models on the Meta Quest 3*, deliberately combines two ideas that are often treated separately. The first is interactive explanation: a learner needs help while acting, not after the task has ended. The second is reality: the explanation must be tied to the state that currently exists in the training environment. In this work, that reality is a live, high-fidelity simulation rather than an unconstrained natural scene. The learner is seated in a Meta Quest 3 virtual-reality setup, operating an F/A-18C Lot 20 in Digital Combat Simulator World (DCS World), and attempting to perform a cold-start procedure without leaving the training flow.

Procedural training creates a different problem from ordinary question answering. The correct next action is not simply a fact stored in a manual. It depends on order, preconditions, cockpit state, previous mistakes, and the learner's current point of confusion. In a cockpit start-up task, a general answer such as "turn on the avionics" is insufficient unless the aircraft has power, the relevant displays are active, and the procedure has reached the proper phase. A foundation model can produce fluent procedural language, but fluency alone does not make an instruction safe, timely, or inspectable. A tutor that explains the wrong state can be worse than a static checklist, because it appears adaptive while silently losing contact with the simulator.

The original proposal for this thesis framed the problem as a Quest 3, in-headset guidance system for sequence-sensitive training [26]. That proposal remains the user-facing motivation: learners should not have to remove the headset, search a PDF, replay a video, or lose spatial context every time they need a hint. The implementation and evaluation developed during the thesis, however, shifted the scientific centre of gravity toward a machine-learning systems question: how can a foundation- model tutor be constrained by evidence from a simulator, by authored procedure rules, and by a domain-adapted vision-language model (VLM) that reports structured cockpit visual facts?

This shift is not a detour from the title. It is the technical condition for making the title credible. In a dynamic, high-load, multimodal VR task, an AI assistant becomes useful only when it can explain the learner's actual task state. The system therefore has to observe enough of that state, bind model outputs to evidence, reject or repair unsupported actions, and preserve a log that can be replayed after the session. This thesis calls that design principle *evidence-grounded interactive explanation*.

1.1.1 What "Reality" Means in This Thesis

The word *reality* in the title does not refer to philosophical realism or to mixed-reality rendering in a narrow graphics sense. It refers to the operational state that the learner is currently acting within. In the Quest 3 setting, this state is multimodal. It includes simulator telemetry, cockpit display pages, authored procedural phase, recent learner actions, retrieved manual context, and the visible overlay that the learner may follow. A tutor that ignores any of these layers can produce an explanation that is locally fluent but practically wrong.

For example, an instruction to continue the FCS BIT sequence is meaningful only if the relevant display page is visible or has been recently observed, the required previous steps are satisfied, and the learner has not already moved past the state being described. A text model can phrase such an instruction, but it cannot be allowed to invent the state that justifies it. A VLM can read display content, but it should not decide the whole procedure. Telemetry can confirm switches and engine parameters, but it does not by itself explain what the learner should do next. "Explain Reality" therefore becomes a systems claim: explanation is produced by connecting these evidence sources rather than by asking a foundation model to improvise a tutor.

The Meta Quest 3 is central because it makes this problem interactive and embodied. In desktop use, a learner can more easily inspect external documents or switch windows. In a headset, help competes with spatial attention, hand movement, cockpit scanning, and task memory. A wrong or stale overlay can draw attention to the wrong object at the wrong time. A correct but unsupported explanation can still encourage over-trust if the learner cannot see why it was given. The thesis therefore treats VR not as a decorative output medium, but as the setting that forces evidence-grounded explanation to be precise, concise, and auditable.

This framing also explains why the work emphasises machine learning rather than only interface design. The system needs a perception component that can read text-rich cockpit displays, a language component that can phrase help from evidence, and

a runtime that prevents either component from exceeding its authority. The ML contribution is not merely the use of a VLM or an LLM. It is the way model outputs are transformed into bounded, checkable claims inside a procedural tutoring loop.

1.2 Case Study: F/A-18C Lot 20 Cold Start in DCS World

The empirical setting is the F/A-18C Lot 20 cold-start procedure in DCS World [7, 8]. The procedure pack used by SimTutor is not an invented prompt script. It is a manually decomposed training guide derived from the DCS World F/A-18C manual and local evaluation materials, represented as an authoritative prior for tutoring and scoring. The current pack contains 33 ordered instructional steps, grouped into phases from initial electrical power through engine start, avionics preparation, flight-control checks, and final pre-taxi configuration.

This task is compact enough to study in a thesis, but demanding enough to expose the evidence problem. Some states are available through DCS-BIOS telemetry, such as switch positions, engine parameters, or indicator values. Other states are visible on cockpit displays. The left and right Digital Display Indicators (DDIs) and the Advanced Multipurpose Color Display (AMPCD) show pages and text that are important for confirming flight-control and navigation states. The tutor therefore cannot rely on a single observation channel. It must combine telemetry, display perception, procedure rules, and retrieved procedural knowledge.

The visual component is intentionally narrow. The VLM is not asked to describe the entire VR scene, infer the learner's intention, or decide which procedure step is complete. It reports a small set of structured visual facts such as whether the FCS page, BIT root page, FCS-MC result, HSI page, or INS alignment text is visible. The procedure engine then interprets those facts together with telemetry and sequence state. This separation between perception and procedure logic is one of the main design lessons of the thesis.

1.3 Problem Statement

The research problem can be stated as follows:

> How can a foundation-model tutor provide interactive procedural explanations inside an immersive simulator while keeping each actionable output

grounded in evidence that can be observed, validated, and replayed?

Three gaps motivate this question. First, manuals, videos, and checklists are authoritative but not state-aware. They can tell the learner what the procedure contains, but they do not know whether the learner has already completed a step, skipped a prerequisite, or reached a display page that needs visual confirmation. Second, simulator telemetry is precise where it exists, but it does not provide a complete explanation channel by itself. It exposes state; it does not decide how to phrase a just-in-time hint or how to explain why a step is blocked. Third, foundation models can generate flexible language and parse visual input, but they require runtime constraints when their outputs may guide procedural action.

The contribution of this thesis lies in connecting these pieces. The language model is not treated as the tutor by itself. It is one component inside a runtime that assembles evidence, evaluates procedure gates, retrieves source material, validates structured outputs, maps overlay targets through allowlists, and logs the entire help cycle. The VLM is similarly constrained: it outputs visual facts rather than high-level procedure decisions. The resulting system is not a generic cockpit autopilot or a general-purpose VR assistant. It is a research prototype for evidence-grounded procedural explanation.

1.4 Research Questions

The thesis is organised around one overarching question and three subordinate research questions.

Main RQ. How can foundation models support interactive explanation in a Meta Quest 3 procedural-training environment when their outputs are grounded in simulator state, procedural knowledge, and structured visual evidence?

RQ1 — Evidence-grounded tutoring runtime. How can simulator telemetry, retrieved procedure knowledge, visual facts, schema validation, overlay allowlists, and replay logging be integrated into a runtime that keeps tutoring outputs state-aware and auditable?

RQ2 — VLM adaptation and visual fact reliability. To what extent can a small-data LoRA-adapted VLM extract structured cockpit visual facts reliably, and how do ontology design and backbone choice affect critical false positives?

RQ3 — Formative Quest 3 pilot evidence. What descriptive evidence does a formative, non-random Quest 3 pilot provide about task completion, procedural errors, workload/trust responses, participant comments, and the boundary between assistance and trustworthy collaboration?

RQ1 is answered through the implemented SimTutor runtime, procedure pack, validation logic, replay/export infrastructure, and live help-cycle evidence. RQ2 is answered through the visual fact ontology, dataset curation pipeline, LoRA training recipe, and base-versus-LoRA holdout benchmarks for Qwen3.5, Qwen3.6, and Gemma. RQ3 is answered through a completed formative pilot with nine participant-coded trials. The RQ3 evidence is descriptive only. It does not establish statistical significance, learning gains, workload reduction, retention, transfer, or faster completion.

1.5 Approach

The approach combines an evidence-grounded runtime with a focused machine- learning adaptation pipeline.

The runtime side represents the F/A-18C cold-start as a declarative procedure pack. Steps, preconditions, completion gates, source priorities, visual fact bindings, and overlay targets are kept outside model prompts. DCS-BIOS telemetry is normalised and aggregated into runtime variables. Retrieved manual and training-guide snippets provide textual grounding for explanations. When a help request occurs, the runtime assembles a structured evidence packet before model generation. The response is then parsed, validated, checked against allowlisted overlay targets, repaired or rejected when necessary, and recorded for replay.

The machine-learning side focuses on cockpit visual facts. The VLM data pipeline captures cockpit display images, applies pre-labelling, supports human review, exports supervised fine-tuning rows, trains LoRA adapters, and evaluates base and adapted backbones on independent holdouts. The key design choice is the move from an early 8-fact ontology to a runtime-aligned 13-fact ontology. The revised ontology removes compound procedure-completion targets and replaces them with local display observations. This makes the VLM's role more modest and more useful: it provides evidence for downstream rules instead of becoming the decision maker.

The pilot side places the system back into the original Quest 3 motivation. Four participants used the tutor and five participants completed passive without-tutor trials. Participant profiles ranged from novice to expert by self-report. The study records

completion state, manual error coding, help cycles, VLM calls, overlay actions, Raw NASA-TLX, usability/trust responses, and open comments. The pilot is included because it tests whether the system can be exercised with real participants in VR and what kinds of failure modes appear. It is not used as a powered human-subject effectiveness study.

1.6 Contributions

This thesis makes four contributions.

1. **An evidence-grounded foundation-model tutoring runtime.** The thesis presents a runtime architecture that connects DCS-BIOS telemetry, procedure-pack gates, retrieval, visual facts, structured response validation, overlay allowlists, deterministic repair/fallback behaviour, and replayable logs. This addresses RQ1 by showing how generated procedural explanations can be constrained by observable evidence rather than model fluency alone.

2. **A visual fact ontology for cockpit evidence.** The thesis develops a 13-fact ontology for F/A-18C cockpit display state. The ontology is designed to separate local visual perception from procedural interpretation. This addresses one of the central risks identified in the 8-fact baseline: compound visual targets can create critical false positives when they ask the VLM to infer procedure completion directly.

3. **A small-data LoRA VLM adaptation and backbone comparison.** The thesis evaluates a Run-003 + Run-005x2 training recipe across Qwen3.5, Qwen3.6, and Gemma backbones. Qwen3.5 is the main result line because it achieves the strongest offline exact-match behaviour on the current holdouts. Qwen3.6 is treated as a strong deployable candidate, while Gemma provides a different conservative error profile. This addresses RQ2.

4. **A formative Quest 3 pilot with bounded human-facing evidence.** The thesis reports a non-random pilot with nine participant-coded trials, including participant experience profiles, completion and error patterns, Raw NASA-TLX, usability/trust scores, and anonymised short comments. This addresses RQ3 by showing feasibility, instrumentation readiness, and assistance-coupled risks without making causal learning claims.

1.7 Thesis Structure

Chapter 2 provides the theoretical background and related work for evidence-grounded tutoring, VLMs, LoRA adaptation, structured visual outputs, simulator instrumentation, and explainable assistance in AR/VR.

Chapter 3 describes the evidence-grounded runtime method: procedure pack design, telemetry processing, retrieval, help-cycle validation, overlay mapping, and replay.

Chapter 4 describes the VLM and pilot methodology: dataset curation, 8-fact to 13-fact ontology evolution, LoRA training, benchmark metrics, backbone comparisons, and Quest 3 pilot protocol.

Chapter 5 documents reproducibility and experimental setup details that support the methods, including provider configuration, artifact export, and figure/table provenance.

Chapter 6 reports the results for RQ1, RQ2, and RQ3 in that order: runtime readiness, VLM adaptation benchmarks, and the formative pilot.

Chapter 7 interprets the findings and limitations, then uses the idea of AI as a trustworthy collaborator in dynamic, high-load, multimodal environments to define future work.

Chapter 8 summarises the contributions and claim boundaries. Appendix 9 contains supplementary tables, charts, command references, and pilot data.

2 Theoretical Background and Related Work

The introduction framed the thesis as an evidence-design problem: a foundation model should explain a live training state only through evidence that the runtime can observe, validate, and replay. This chapter provides the theoretical and related-work background needed for that claim. It first introduces the procedural training setting and the observation problem created by DCS World. It then reviews the machine-learning concepts used in the thesis: VLMs, structured visual outputs, LoRA-based parameter-efficient adaptation, retrieval grounding, and reliability metrics. The final sections position the work against intelligent tutoring systems, simulator instrumentation, and grounded AI assistance in AR/VR.

2.1 Procedural Training in Immersive Simulation

Procedural training differs from factual instruction because success depends on ordered action under state constraints. A learner must know not only what an action is, but when it is permissible, how to check that it has taken effect, and how to recover when an intermediate state differs from expectation. In aviation and industrial settings, these requirements are typically externalised through manuals, checklists, instructors, and simulator procedures. Intelligent Tutoring Systems (ITS) research has long treated explicit task models and learner-state tracking as central to effective tutoring [2, 17, 22]. The challenge in this thesis is that the task environment is not built as a tutor. It is a high-fidelity simulator that must be instrumented.

Virtual reality increases the need for in-context help. A learner using a Meta Quest 3 in DCS World can remain spatially present in the cockpit, but that presence is fragile. Searching a PDF, replaying a video, or asking an external chatbot breaks task flow. AR and VR guidance research has shown the value of placing instructions in the user's task context, especially in maintenance and procedural settings [18]. The thesis builds on

that premise but adds a stricter requirement: if help is generated at runtime, it must be grounded in the current simulator state rather than presented as a generic overlay.

2.1.1 From Checklist Following to State Estimation

The cold-start task is easy to describe as a checklist and difficult to support as an interactive system. A printed checklist assumes that the learner can map a line of text to a cockpit region, execute the action, observe the result, and decide when to continue. A tutor that intervenes during the task has to perform part of that mapping explicitly. It must estimate which step is active, which preconditions are already satisfied, which completion evidence is missing, and whether the learner's most recent action moved the system toward or away from the expected state.

This is a state-estimation problem rather than a pure natural-language problem. The procedure text alone is insufficient, because the correct explanation depends on the simulator state. Conversely, telemetry alone is insufficient, because a numerical or boolean simulator variable has to be interpreted through the authored procedure. The thesis therefore treats tutoring as an interaction between three representations: the authored procedural prior, the observed simulator state, and the generated explanation. The foundation model operates only in the third representation and receives the first two as evidence.

This framing is also important for evaluation. A generic tutoring system might be evaluated by answer fluency or student satisfaction. A state-estimation tutor also has to be evaluated by whether its evidence contracts are correct, whether it can distinguish observed false states from unobserved states, and whether it avoids advancing the learner on weak evidence. Those concerns explain why this thesis gives substantial space to telemetry variables, visual fact ontologies, critical false positives, and replay logs. They are not incidental engineering details; they are the technical substrate that makes an interactive explanation about the current reality meaningful.

2.2 DCS World, F/A-18C Lot 20, and Observable State

DCS World provides a high-fidelity simulation platform with detailed aircraft systems and VR support [7]. The case study used here is the F/A-18C Lot 20 cold-start procedure [8]. The procedure is suitable for this thesis because it is bounded, repeatable, and strongly state-dependent. The tutor must reason about electrical power, APU and

engine state, display power, communications setup, navigation alignment, flight-control checks, and final pre-taxi settings.

The procedure pack used by SimTutor is a manually decomposed training guide. It converts the DCS World F/A-18C manual and local evaluation materials into 33 instructional steps with preconditions, completion gates, UI targets, and evidence requirements. This pack is a form of authored prior knowledge. It does not replace the simulator; it defines what the tutor is allowed to treat as a valid procedural state.

The observation problem has two main channels. DCS-BIOS exposes many cockpit states through a telemetry interface [6]. It is the preferred source for switch positions, engine parameters, and several indicator values. However, cockpit display content is not fully represented as simple telemetry. The left and right Digital Display Indicators (DDIs) and the Advanced Multipurpose Color Display (AMPCD) show pages and text that matter for the cold-start. This is why the thesis uses a VLM for selected display facts while keeping procedural interpretation in deterministic rules.

2.3 Foundation Models as Bounded Components

Transformer-based foundation models provide the representational and instruction-following basis for modern language and multimodal systems [23]. In educational applications, large language models can generate flexible explanations, but they also introduce risks: unsupported confidence, hallucinated references, and fluent but context-free feedback [13]. The design choice in this thesis is therefore not to ask a foundation model to be an unconstrained tutor. Instead, the model receives an evidence packet and returns a structured response that the runtime can validate before display.

This view aligns with broader work on grounding model output in external evidence or action constraints. Retrieval-Augmented Generation (RAG) conditions language generation on retrieved documents rather than relying only on parametric memory [14]. Work on grounded robotic language use, such as SayCan, similarly separates linguistic plausibility from action feasibility [1]. SimTutor adapts this principle to simulator tutoring: an answer may be linguistically plausible, but it is not actionable unless its step, explanation, and overlay target are supported by telemetry, procedure gates, retrieved sources, or visual facts.

2.3.1 Grounding, Authority, and Explanation

Grounding is used in several ways in the literature. In retrieval-augmented generation it often means that a model response is conditioned on external documents. In embodied language systems it can mean that a proposed action is feasible in the physical or simulated world. In explainable AI it can mean that the system exposes reasons for a recommendation. This thesis combines those meanings but keeps them operationally separate. Retrieved text grounds procedural wording; telemetry grounds simulator state; visual facts ground display observations; and the harness grounds actionability.

The separation matters because a single natural-language explanation can hide several different authority claims. When a tutor says "press this button next", it may be claiming that the learner is on a specific step, that the previous step is complete, that a display cue has been observed, that the button is a valid target, and that the recommended action is safe within the procedure. If all of these claims are merged into one model-generated sentence, a fluent mistake is hard to diagnose. If each claim is tied to a distinct evidence type, the runtime can validate the parts before the learner sees the result.

For that reason, the thesis uses explanation in a stricter sense than ordinary help text. An explanation should make the model's response inspectable against evidence. It should be possible to ask: which telemetry variable, gate failure, retrieved sentence, or visual fact supports this guidance? If the evidence is missing, the response should expose uncertainty instead of creating a confident story. This is the theoretical reason for treating the foundation model as a bounded component. The model is useful because it can combine context and phrase help flexibly; it is bounded because the procedure state, target space, and evidence references must remain outside its sole control.

2.4 Vision-Language Models and Structured Visual Facts

Vision-language models extend foundation-model capabilities to image-text tasks. General VLM work such as Qwen-VL, Qwen3-VL, LLaVA, and InstructBLIP shows how visual encoders and language decoders can support image reasoning, visual question answering, instruction following, and text-rich perception [4, 5, 15, 19]. The cockpit task in this thesis uses these capabilities in a narrow way. The VLM is not asked for a free-form description of the cockpit. It is asked to return a typed JSON object containing a fixed set of visual facts.

The visual fact representation has three states: `seen`, `not_seen`, and `uncertain`. A fact such as `fcsmc_final_go_result_visible` is a local claim about a visible display cue. It is not a claim that the whole FCS BIT step is complete. That distinction is central. The first 8-fact ontology contained more compound targets and exposed a recurring failure mode: the model could report a procedure-completion-like fact as `seen` even when the human label was `not_seen`. The 13-fact ontology used by the current runtime decomposes such targets into visual atoms.

This design also relates to screen and GUI understanding. Screen-content models must reason about fixed regions, symbolic text, and UI layout rather than ordinary natural images [25]. Cockpit displays share that structured quality. The thesis therefore evaluates not only aggregate accuracy but also per-fact behaviour and critical false positives, because a false positive on a completion-type fact may advance the tutoring state at the wrong time.

2.4.1 Structured Output as Visual Prediction

The VLM task is best understood as structured prediction over a small ontology. For each image, the model predicts a vector of categorical states rather than a caption. If the ontology contains $K = 13$ facts and each fact has three states, the target is a fixed-length output in a constrained label space. The JSON format is a convenient interface, but the learning problem is not merely JSON generation. The model has to align visual evidence with fact definitions, distinguish page labels from actual pages, separate transient intermediate states from final states, and decide when a cue is too ambiguous for a `seen` or `not_seen` decision.

Structured output changes the error surface. A free-form caption can be partially useful even if it omits a cue. A structured fact vector is either usable by downstream code or rejected by schema validation, and each fact state can affect a procedure gate differently. This is why JSON validity and schema validity are not cosmetic metrics. A model that understands the image but returns a malformed object cannot be integrated safely into the runtime. A model that returns valid JSON but hallucinates a completion fact is even more dangerous, because the output looks machine-readable while the underlying state claim is wrong.

The cockpit domain also differs from common VLM benchmarks. Many benchmark images contain natural objects and open-ended descriptions. The cockpit images in this thesis are text-heavy, low-context, and region-structured. A small visual cue may be a pushbutton label, an option name, a page title, a row in a BIT result table, or a

transient alignment message. The ontology therefore acts as a bridge between perception and control logic. It tells the model which propositions matter and tells the runtime how to consume them. The quality of that ontology is as important as the capacity of the backbone model.

2.4.2 Uncertainty as a First-Class State

The `uncertain` state is not a decorative third class. It is a mechanism for preserving epistemic caution in a pipeline that eventually affects user guidance. In a binary ontology, every ambiguous cue must be forced into `seen` or `not_seen`. That can inflate false positives when the image is blurry or the display is partially occluded, and it can inflate false negatives when a cue is present but not readable enough for a confident label. By allowing `uncertain`, the VLM can report that the image does not support a clean claim.

This design only helps if the downstream runtime respects it. In SimTutor, `uncertain` is not treated as completion evidence. It can be logged, surfaced to the language model as an uncertainty cue, or used to trigger a fresh observation, but it should not silently satisfy a procedure gate. This choice aligns the ML representation with the tutoring safety boundary: when the perception layer is unsure, the explanation should not pretend otherwise. Future temporal VLM work can reduce uncertainty by considering multiple frames, but the current single-frame method needs uncertainty as an explicit output state.

2.5 LoRA and Parameter-Efficient VLM Adaptation

Full fine-tuning of large multimodal models is often unnecessary or impractical for narrow domain tasks. Low-Rank Adaptation (LoRA) freezes the base model weights and learns low-rank update matrices for selected modules [12]. If a base weight matrix W is frozen, LoRA represents the update as:

$$W' = W + \Delta W, \qquad \Delta W = BA,$$

where A and B are low-rank trainable matrices. This reduces the number of trainable parameters while preserving the base model. Parameter-efficient fine-tuning libraries such as PEFT and TRL provide practical tooling for this workflow [16, 24], while Unsloth is used in the local training reports for efficient VLM fine-tuning [10].

In this thesis, LoRA is used for a structured output task: image-to-facts JSON. The

training examples pair a cockpit composite image with a target JSON schema containing fact states. The goal is not to make the model more generally knowledgeable about aviation. The goal is to make the model more reliable at a small set of display-state recognitions that the runtime needs. This is why the thesis treats ontology design, data curation, and holdout independence as part of the ML method rather than as peripheral engineering details.

2.5.1 Why PEFT Fits This Domain

The data regime in this thesis is closer to domain adaptation than to broad model training. The cockpit facts are specialised, the dataset is small, and the desired output format is narrow. Full fine-tuning would introduce several problems: it would require more compute, increase the risk of overfitting, make model distribution harder, and blur the boundary between the base model and the domain-specific adaptation. A LoRA adapter is a better fit because it stores the task-specific update separately from the base model and can be evaluated, replaced, or disabled as a distinct artifact.

PEFT also matches the thesis's claim structure. The work does not claim to create a generally better VLM. It claims that a small adapter can improve one structured perception task when the ontology and data are aligned with the runtime. Keeping the adaptation small makes this claim more transparent. If a later procedure needs a different visual ontology, a new adapter can be trained and benchmarked without redefining the whole tutoring architecture. If a model family changes, the same data recipe can be tested on the new backbone, as done with Qwen3.6 and Gemma in the evaluation.

There is also a practical reproducibility argument. Adapter training produces a manageable artifact that can be paired with benchmark JSON, per-sample predictions, and error reports. The adapter is not the only artifact that matters, but it is a compact representation of the learned part of the method. This makes the ML contribution easier to audit: the reader can separate the authored procedure pack, the reviewed labels, the training recipe, the adapter, and the holdout metrics.

2.5.2 Instruction Tuning and Schema Learning

The SFT rows used in the thesis teach two things at once. First, they teach the visual decision boundary for each fact. Second, they teach the model to emit the required JSON contract. These two objectives interact. A model may know that a page is visible

but fail the runtime contract if it omits a fact, adds an unsupported field, or uses a state outside the allowed vocabulary. Conversely, a model may learn the schema while still making visual mistakes. This is why the evaluation separates JSON validity, schema validity, and fact-level accuracy.

The bilingual English/Chinese export is also best viewed as instruction-format augmentation rather than new visual evidence. Each reviewed image contributes the same fact labels under two instruction variants. This can make the adapter less brittle to prompt language and target phrasing, but it does not create new cockpit states. The thesis therefore treats bilingual rows as useful supervision for interface robustness while still relying on independent holdout images for generalisation evidence.

2.6 Metrics for Reliability: Beyond Aggregate Accuracy

The VLM evaluation uses several metrics because no single number captures the risk profile of a structured visual fact extractor. JSON validity and schema validity measure whether the model returned a usable object. Fact accuracy counts individual fact-state decisions. Macro F1 balances state classes across facts. Seen F1 focuses on the positive visual-detection state, which is often the state that confirms a cockpit display cue. Sample exact match is stricter: all facts in an image must be correct. Critical false positives count high-risk cases where the model predicts `seen` for facts that should not be considered visible.

The last metric is especially important in a tutor. A false negative may cause the system to ask for another check. A false positive can cause it to treat a display state as achieved when it has not been achieved. The evaluation therefore interprets critical false positives separately from aggregate accuracy. This is one reason Gemma remains interesting as a comparison backbone: its adapted line is more conservative for some critical facts, even when its overall exact-match performance is lower than Qwen3.5.

2.6.1 Reliability Metrics in a Downstream System

The choice of metric depends on the downstream use. If the VLM were only used to summarise screenshots for a human analyst, macro F1 or qualitative caption quality might be enough. In this thesis the output can become part of a runtime evidence packet. That makes asymmetric errors important. A missed `seen` label may cause the tutor to ask the learner to check again, but a false `seen` label on a completion-critical

fact can allow an unearned step transition or an incorrect explanation. This asymmetry motivates the separate critical-false-positive count.

Sample exact match is also unusually strict but informative. The tutor does not consume only one fact in isolation; a help cycle may include a whole visual snapshot. If one of thirteen facts is wrong, the snapshot is not perfectly aligned with the reviewed label. Exact match therefore gives an intuitive measure of whether the entire visual state is clean enough for downstream reasoning. It should not replace fact-level analysis, because some facts are more important than others, but it exposes whether aggregate accuracy is hiding multi-fact residual errors.

Finally, schema validity deserves attention because structured model output is a systems contract. In an offline benchmark, an invalid schema is an error. In a live tutor, it is also a control-flow event: the runtime has to reject, repair, or fall back. The ML evaluation therefore feeds directly into the RQ1 runtime design. Reliable perception is not only about high accuracy; it is about producing outputs that the system can parse, validate, and safely combine with other evidence.

2.7 Grounded and Explainable Assistance in AR/VR

Explainable AI research argues that users and developers need to understand why a system produced a recommendation, especially when the recommendation affects action [9]. Human-AI interaction guidelines similarly emphasise intelligibility, appropriate trust, and graceful handling of uncertainty [3]. In this thesis, explanation is not only a textual feature. It is a runtime property: every actionable output should have an evidence path.

The Meta Quest 3 setting makes this requirement concrete. A learner in VR has limited attention and limited tolerance for checking external material. The tutor must therefore be concise, but concision is not enough. It must also be clear why a highlighted control, a blocked step, or a recovery hint is being shown. SimTutor operationalises this through evidence references, response schemas, overlay allowlists, and replayable help-cycle logs.

2.8 Positioning of This Thesis

The thesis sits between ML adaptation, tutoring systems, simulator instrumentation, and HCI/VR evaluation. From ITS research, it takes the need for explicit task structure.

From RAG and grounded model work, it takes the principle that generated output should be conditioned on external evidence. From VLM research, it takes the ability to parse text-rich cockpit display images. From LoRA and PEFT, it takes the practical method for adapting large models to a narrow task. From AR/VR guidance, it takes the user-facing goal of keeping help inside the task environment.

The novelty is the integration and the boundary discipline. The system does not claim to solve general cockpit autonomy, general VR tutoring, or general learning effectiveness. It shows how a foundation-model tutor for one high-fidelity VR procedure can be made more auditable by tying generated explanations to telemetry, authored procedure rules, retrieved sources, and structured visual facts. The remaining chapters turn that position into a method, a set of VLM benchmarks, and a formative Quest 3 pilot.

3 Methods: Evidence-Grounded Tutoring Runtime

This chapter answers RQ1 at the method level. It describes how SimTutor turns foundation-model assistance into an evidence-grounded help cycle for the F/A-18C Lot 20 cold-start task. The chapter is not organised as an implementation inventory. Instead, it describes the contracts that matter for interactive explanation: procedure modelling, telemetry grounding, retrieval, visual fact integration, response validation, overlay execution, and replay.

3.1 Design Requirements

The runtime was designed around six requirements.

1. **State-aware help.** The tutor must reason from the current simulator state, not only from a static checklist.
2. **Evidence separation.** Telemetry, visual facts, retrieved text, and procedure rules must remain distinguishable after generation.
3. **Bounded model authority.** The language model may phrase explanations, but it must not silently define the procedure state or invent overlay targets.
4. **Inspectable visual perception.** The VLM should output structured facts that can be inspected and scored, not free-form cockpit narratives.
5. **Replayability.** A help decision must be reconstructable from logged observations, evidence packets, model output, validation decisions, and overlay actions.
6. **Pilot-readiness.** The runtime must generate participant-level artifacts for later analysis without turning the formative pilot into a hidden manual coding exercise.

These requirements follow directly from the thesis title. Interactive explanation in VR is only defensible when the explanation can be traced to the simulated reality it describes.

3.2 Evidence Object Model

The runtime treats evidence as typed objects rather than as unstructured prompt text. This is a design choice with methodological consequences. A telemetry variable, a failed gate, a retrieved sentence, a visual fact, and a recent action are all useful to the tutor, but they should not have the same meaning. If they are collapsed into one paragraph before generation, the model can produce a fluent answer whose support is impossible to reconstruct. If they are kept as typed entries, the runtime can later check whether an overlay or diagnosis refers to evidence that actually existed.

The evidence object model therefore records four properties for each usable piece of context. First, it records the evidence type, such as `var`, `gate`, `rag`, `delta`, or `visual`. Second, it records a stable reference that can be used in prompts, validation, and logs. Third, it records the observed value or textual support. Fourth, it records availability and quality metadata where relevant, such as source-missing status, frame age, or retrieval source. This structure makes the help response auditable because a model cannot simply cite an arbitrary reason; the cited reason has to correspond to an evidence reference in the current packet.

The distinction between evidence and interpretation is deliberate. A visual fact can state that `fcsmc_final_go_result_visible` is `seen`; it should not state that the whole FCS BIT procedure is complete. A telemetry variable can state that the flap switch is in AUTO; it should not decide whether the learner understands why AUTO is required. The procedure engine and tutor response combine evidence into an explanation, but the raw evidence remains inspectable underneath that explanation.

3.3 Procedure Pack as Authoritative Prior

The procedure pack is the authored domain prior for the tutor. It represents the F/A-18C cold-start as 33 instructional steps derived from the DCS World manual and local training materials. Each step has a stable identifier, phase, preconditions, completion gates, evidence requirements, and UI targets. The pack is stored as configuration rather than embedded in prompts. This keeps the procedure auditable and separates domain

knowledge from model wording.

The pack does three things during a help cycle. First, it constrains which steps may be considered next. A step cannot be recommended as available when its preconditions are unsatisfied. Second, it specifies the evidence used to confirm progress. Some steps are telemetry-dominant; display-dependent steps use visual fact bindings; other checks degrade conservatively when the required evidence is not available. Third, it defines the overlay target space. The model may suggest help, but the runtime maps visual highlights only to known cockpit targets.

This design avoids a common failure mode of prompt-driven tutors. If the procedure is represented only in natural language, the model can blend steps, skip preconditions, or invent target names. In SimTutor, the prompt receives a runtime-generated evidence packet, but the procedure itself remains a declarative object controlled by the system.

3.4 Authority Hierarchy

The runtime uses an authority hierarchy to decide what can override what. The procedure pack has authority over step order, allowed targets, and evidence requirements. Telemetry has authority over directly exported simulator state. Reviewed visual facts have authority only over the defined visual propositions and only within their expiry and quality constraints. Retrieved text supports explanation but does not override gates. The LLM phrases guidance and may propose a diagnosis, but it does not define the final procedure state.

This hierarchy prevents several subtle failures. Without it, a retrieved manual snippet could suggest a procedure variant that conflicts with the active airfield cold-start pack. A VLM could infer that a step is complete because it sees a related page, even when a required telemetry gate is false. An LLM could choose a plausible cockpit control that is not in the current UI allowlist. The hierarchy resolves these conflicts before user-facing output is produced: the stricter local contract wins over a broader or less reliable evidence source.

The hierarchy also supports graceful degradation. If a visual fact is missing, the runtime does not have to abandon the whole help cycle. It can still use telemetry, gates, recent actions, and retrieved procedure text. The generated response is simply constrained to what those evidence sources can support. In practice, this means that the tutor can say that visual confirmation is unavailable, can ask the learner to verify a display, or can give a text-only hint without highlighting a target. The user sees less

ambitious help, but not invented certainty.

3.5 Telemetry Grounding

DCS-BIOS telemetry is the primary observation channel for simulator state. The runtime receives raw frames, normalises exported values, derives higher-level variables, and tracks missing sources separately from false values. This distinction is important. A variable can be false because the cockpit state is not satisfied, or it can be unavailable because the source was not exported or not observed in the current frame. Treating both cases as the same would make the tutor overconfident.

The telemetry pipeline also performs delta sanitisation and aggregation. Raw simulator signals can jitter, repeat, or change too quickly for a model prompt. The runtime therefore turns high-frequency state changes into stable variables and concise recent-action summaries. These summaries help the tutor explain what changed without exposing the model to an unbounded log.

Examples of telemetry-derived evidence include battery and generator state, APU readiness, engine RPM thresholds, throttle movement, INS mode, radar mode, flap state, parking-brake release, IFEI BINGO fuel settings, and attitude-source selection. Where telemetry is authoritative, the VLM is not used. This is a deliberate design choice: computer vision should not replace direct simulator state when direct state exists.

3.6 Retrieval and Source Policy

Retrieved procedural knowledge supports the natural-language explanation part of a help response. The runtime builds grounding queries from the current procedure candidate and simulator state, retrieves local manual or training snippets, and supplies them to the model as source material. A source policy restricts which documents and chunks are allowed. This keeps the model from receiving arbitrary repository text or unrelated simulator material.

Retrieval is therefore used for explanation, not for uncontrolled decision making. The procedure pack and telemetry decide whether a step is available or blocked; retrieved text helps phrase the action, check, and rationale. This division mirrors the thesis's broader boundary: foundation models should explain evidence, not replace the evidence.

3.7 Visual Fact Integration

Visual facts enter the help cycle when display content cannot be read reliably from telemetry. The runtime uses a fixed composite panel layout covering the left DDI, right DDI, and AMPCD. The VLM receives one or more candidate frames and returns a JSON object with the allowed fact identifiers and states. The current 13-fact ontology includes page visibility facts, FCS page markings, FCS-MC BIT states, HSI/map-layer facts, and INS alignment text facts.

The runtime treats these facts as observations, not as procedure decisions. A visual fact can satisfy a step binding only when the procedure pack says it is relevant. Some facts are short-lived and expire quickly; others, such as a final GO cue, may be sticky for a longer time because the display state can disappear after the learner changes pages. The sticky mechanism prevents the system from forgetting transient but important visual evidence, while still keeping the evidence explicit.

This architecture is what makes VLM benchmark performance relevant to the runtime. A higher fact accuracy matters because the downstream rule engine uses those facts as evidence. A critical false positive matters because it can make a completion-like visual cue appear present when it is absent. The runtime therefore logs visual fact calls and preserves their outputs for replay and error analysis.

3.8 Help Cycle

A help cycle begins when the learner requests assistance, for example through the configured hotkey path used in the pilot. The runtime then performs the following sequence:

1. Read the latest telemetry frame and derived variables.
2. Merge recent actions and any available visual fact observations.
3. Evaluate procedure preconditions and completion gates.
4. Select the current blocked, pending, or next candidate step.
5. Retrieve source snippets under the local source policy.
6. Build a structured prompt containing the evidence packet.
7. Request a model response.

8. Parse the response against the expected schema.
9. Validate overlay targets and evidence references.
10. Execute, repair, or suppress overlay actions.
11. Record the observation, model response, validation result, and action in the event log.

The model is therefore invoked only after evidence assembly. The response is used only after validation. This sequencing is the runtime expression of "Explain Reality": the model explains a bounded evidence packet and the system checks whether the explanation can be safely displayed.

3.9 Structured Response and Overlay Validation

The tutor response is structured rather than free-form. It contains fields for diagnosis, recommended action, checks, explanation, and overlay targets. The runtime parses the response, validates it against a schema, and maps overlay targets through an allowlist. If a model proposes an unsupported target, a coarse target, or an action that lacks evidence, the runtime can repair the output, fall back to text-only guidance, or suppress the overlay.

This validation layer matters for both safety and evaluation. It prevents a model from directly controlling visual highlights through arbitrary text. It also creates a record of how often model outputs required repair. In the pilot reported later, all executed overlay actions passed validation; fallback events were mainly repair or mapping events rather than full deterministic-only responses. That result supports RQ1 at the system level, while also showing where the model still needed runtime correction.

3.10 Uncertainty and Degradation Modes

The runtime has to handle uncertainty as a normal operating state. In a live simulator, a help request can arrive while a switch is moving, while a display page is changing, while a DCS-BIOS variable is unavailable, or while the VLM frame is stale. Treating these cases as ordinary false values would make the tutor overconfident. Treating them as hard failures would make the system too fragile for live use. The method therefore defines degradation modes.

The first mode is *evidence missing*. A required source is not available, or the source is known to be outside the current observation contract. The runtime may still provide general procedure guidance, but it should not claim that the state has been checked. The second mode is *evidence conflicting*. For example, a visual fact and a telemetry-derived step candidate may point to different interpretations. In that case the stricter authority hierarchy and harness rules determine whether the response is repaired, downgraded, or held. The third mode is *output invalid*. The model response fails schema, target, or evidence-reference validation, so the runtime falls back to a deterministic or text-only response.

These modes matter because they make uncertainty visible in logs and, where appropriate, in user-facing language. A tutor that cannot tell the difference between "you did not do this" and "I cannot observe this" will mislead the learner. SimTutor's design keeps those cases separate. This is especially important in the F/A-18C cold-start because some steps combine slowly changing simulator state, transient display text, and manual cockpit navigation. The runtime can be helpful only if it knows when its own evidence is incomplete.

3.11 VR Interaction Boundary

The Meta Quest 3 setting affects the runtime design even though this thesis is not primarily a VR interface study. In a desktop setting, a user can more easily switch between a simulator, a manual, a chat window, and logs. In a headset, that switching cost is much higher. Guidance must be brief, spatially useful, and timed to the learner's current action. At the same time, VR guidance can become intrusive if it highlights too many targets, uses stale state, or interrupts a learner who is already completing the action.

The runtime therefore treats overlay guidance as a constrained output channel, not as decorative UI. Highlighted targets must come from the procedure pack's UI map. Multi-target guidance is bounded. Evidence references are required for overlay plans. When the model proposes a target that is too coarse or not grounded enough, the harness may repair it or suppress the overlay. The language part of the response remains available, but spatial guidance is held to a stricter standard because it directly shapes the learner's attention in the cockpit.

This boundary also explains why the pilot logs matter. A successful VR trial is not only a participant reaching the last step. It is also a sequence of help requests, VLM calls,

repairs, highlights, and state transitions that can be inspected afterward. Without that record, it would be difficult to tell whether the tutor helped because its evidence loop worked, because the participant was already experienced, or because a human observer filled gaps informally. The exported help-cycle artifacts make that distinction analyzable in future work.

3.12 Logging, Replay, and Export

Every help cycle and trial-level event is written to logs that can be replayed and exported. Replay is important because a live VR session is difficult to inspect in the moment. The export pipeline produces trial summaries, per-step coding files, help-cycle tables, action timelines, quality gates, and session metadata. These artifacts support both regression testing and pilot analysis.

The replay design also protects the interpretation of results. A benchmark metric by itself does not show how the tutor behaved in a live trial. A pilot completion table by itself does not show whether the overlay path was valid. By preserving evidence packets, model outputs, validation decisions, and participant-level exports, the thesis can connect ML reliability to the actual conditions under which a user saw help in VR.

3.13 Method Boundary

The runtime method has a clear boundary. It is not a certified flight-training system, not a general cockpit automation system, and not a proof of learning effectiveness. It is a research prototype for studying how evidence-grounded foundation-model explanations can operate inside an immersive simulator. The next chapters evaluate the two most important parts of that prototype: the VLM visual evidence layer and the formative Quest 3 pilot.

4 Methods: Visual Fact Adaptation and Pilot Protocol

Chapter 3 presented the runtime architecture that keeps tutor output downstream of evidence and upstream of validation. This chapter describes how the remaining empirical evidence is produced, curated, and evaluated. The method is organised around a practical question: how can a small set of reviewed cockpit images, structured labels, and formative VR sessions become auditable evidence for a machine-learning tutoring system?

The chapter has three roles. First, it records the runtime evidence interfaces used in later replay and pilot analysis: telemetry variables, gating diagnostics, retrieved snippets, and deterministic fallback. Second, it presents the visual fact methodology for RQ2: screenshot capture, pre-labelling, human review, ontology revision, bilingual SFT export, LoRA adaptation, and holdout benchmarking. Third, it documents the formative VR pilot protocol for RQ3. The pilot is treated as descriptive evidence about feasibility and interaction patterns; it is not used for inferential claims about learning gains.

4.1 Procedure Runtime and Telemetry Grounding

Telemetry and procedure gates provide the method-level state evidence on which the help cycle depends. The architectural placement of this layer was described in Chapter 3; the methodological concern here is how raw simulator observations become stable, inspectable variables for later benchmarking, replay, and help-cycle analysis.

4.1.1 Variable Resolution and Source-Missing Tracking

The procedure pack declares a telemetry map that projects raw DCS-BIOS fields onto normalised runtime variables. The resolver produces a flat dictionary of key–value pairs

and separately tracks unavailable sources in `vars_source_missing`. This separation is important because a false value and an unobserved value should not have the same methodological meaning. A switch known to be off can support corrective guidance. A missing source supports only uncertainty-aware guidance or fallback.

4.1.2 Gating Rule Evaluation

Each procedure step contains precondition and completion gates. The gate DSL supports threshold checks, range checks, boolean flags, and elapsed-time conditions. Missing, source-missing, or sentinel values fail with diagnostic reasons rather than being silently coerced. The failed gate and reason are included in the help-cycle context and can later be inspected in logs. This turns procedure progress into a rule-governed evidence claim rather than a model-inferred narrative.

4.1.3 Delta Sanitisation and Aggregation

Raw telemetry can change at high frequency because of simulator physics, instrument jitter, and transitional switch states. Delta sanitisation applies per-variable thresholds, debounce windows, and filtering rules. Delta aggregation then compresses recent, sanitised changes into a prompt-safe summary. The resulting deltas are used as evidence of recent learner action without exposing every raw simulator frame to the language model.

4.1.4 Deterministic Fallback

Fallback is part of the method rather than only an implementation safeguard. When model output is unavailable or invalid, the runtime computes a conservative hint from the active step, unsatisfied gates, recent actions, and evidence availability. The fallback distinguishes hard blockers, where observed evidence shows that a gate is unsatisfied, from soft blockers, where required evidence is missing. Overlay guidance is suppressed unless a local rule identifies a unique valid target.

4.2 Knowledge Grounding and Source Policy

Retrieved procedural knowledge supplies textual context for generated help, but it is deliberately source-controlled. The local retrieval adapter indexes curated procedure and aircraft documentation at sentence level and ranks snippets with BM25 [20]. A

grounding query is constructed from the active step, its description, and the current observation; the top snippets are included in the help-cycle context.

A declarative source policy restricts retrieval to allowed document sources. This prevents the model from receiving procedure variants outside the active procedure pack, such as carrier-specific instructions when the current profile is an airfield cold start. In the thesis method, retrieval is therefore not a general document search facility. It is a controlled evidence channel whose scope is part of the procedure configuration.

4.3 Visual Fact Data Pipeline

The visual fact pipeline is the main methodology for RQ2. It turns DCS World cockpit screenshots into supervised examples and then into a benchmarked visual fact extractor. The pipeline is designed for traceability: a benchmark error should be traceable back to the model prediction, the human-reviewed label, the pre-label, and the original image.

Figure 4.1 shows the pipeline as a loop. Holdout failures are not only final results; they can reveal weaknesses in the ontology and guide later data collection.

1. **Screenshot capture.** DCS World renders the left DDI, AMPCD, and right DDI into one composite-panel image. The same composite layout is used for data collection, pre-labelling, training, benchmarking, and runtime inference. This keeps the input distribution stable across the method.

2. **VLM pre-labelling.** A large external VLM produces initial labels and evidence notes for each image. Pre-labelling is used as an annotation aid, not as ground truth.

3. **Human review.** Pre-labelled samples are reviewed in Label Studio [21]. The reviewer corrects fact states and evidence notes while preserving a link to the raw image and pre-label.

4. **Reviewed JSONL export.** The reviewed export is the authoritative ground-truth artifact for downstream training and evaluation.

5. **Bilingual SFT generation.** Each reviewed image yields an English and a Chinese training row with the same image and fact labels. The assistant target is structured JSON containing visual facts and a short summary; external management fields such as frame identifiers, file paths, and confidence fields are excluded from the target.

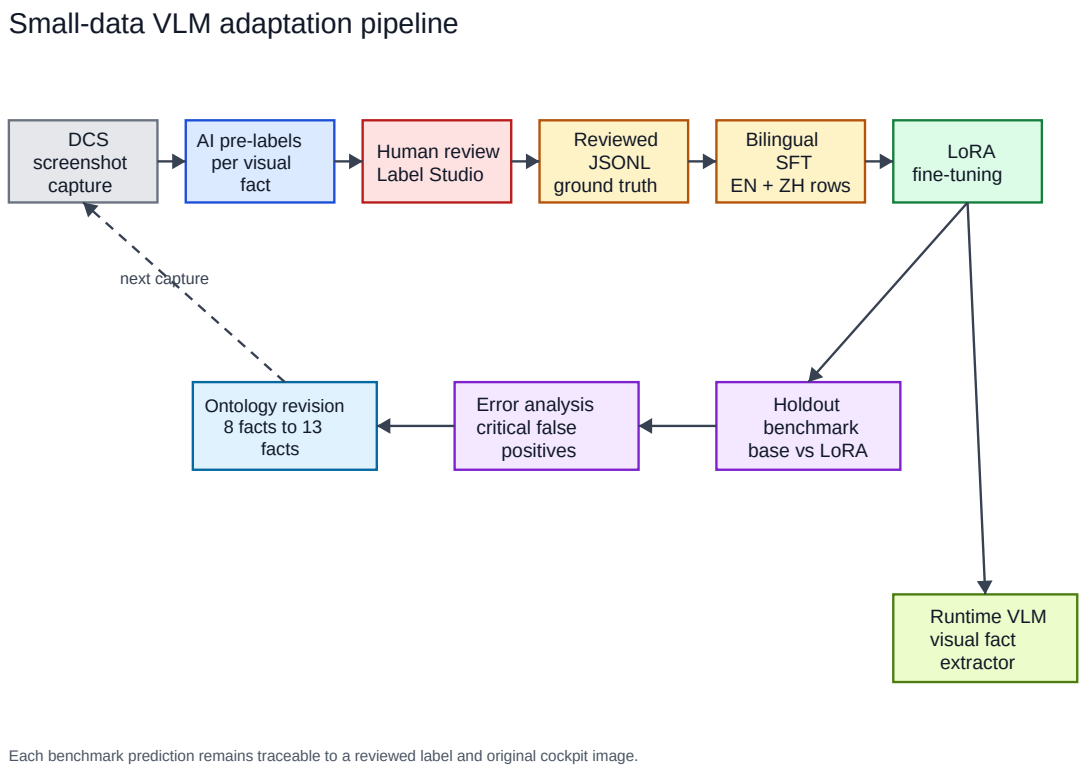


Figure 4.1: VLM adaptation pipeline. DCS World composite screenshots are pre-labelled, human-reviewed, converted into bilingual supervised fine-tuning rows, adapted with LoRA, evaluated on independent holdout sets, and used to drive ontology revision.

6. **LoRA adaptation.** The SFT rows are used to train LoRA adapters for VLM backbones [12]. The implementation uses PEFT adapter definitions [16], TRL supervised fine-tuning infrastructure [24], and Unsloth acceleration for 4-bit VLM training [10].

7. **Holdout benchmark and revision.** Base and LoRA configurations are evaluated on independent holdout sets. Errors are inspected at fact level, especially for completion-critical facts, and the resulting failure modes feed back into ontology and dataset revisions.

The method intentionally keeps the VLM's task narrow. The model is trained to report visual facts with states `seen`, `not_seen`, or `uncertain`. It is not trained to decide procedure completion or produce tutor actions. Those decisions remain in the procedure engine and harness.

4.3.1 Annotation Unit and Decision Boundaries

The annotation unit is one composite-panel screenshot paired with the complete 13-fact label vector. A sample is not labelled only for the fact that triggered collection; every fact in the ontology is reviewed. This full-vector labelling is important because the benchmark later evaluates complete visual snapshots. It also prevents the training data from becoming a set of isolated positive examples with poorly defined negatives.

Reviewers apply fact-specific decision boundaries. A page option label is not the same thing as the page itself. A visible FCS-MC entry on a BIT root page is not the FCS-MC subpage. A PBIT GO status is not the same as the final FCS-MC GO result. HSI symbology without a coloured map layer is not sufficient for the map-layer fact. INS OK must be clearly readable near the alignment block rather than inferred from the elapsed procedure phase. These rules are written into the benchmark prompt and reflected in the ontology, so the model is trained and evaluated against the same conceptual contract.

The `uncertain` label is used when the screenshot does not support a clean decision. This includes blurred frames, partially obstructed display regions, ambiguous text, or states that require temporal history rather than a single image. The reviewer is not asked to guess what the simulator probably did before or after the screenshot. That discipline keeps the VLM task aligned with the runtime boundary: single-frame perception reports visible facts, while procedure reasoning happens downstream.

Pre-labelling accelerates the process but does not define ground truth. The external VLM's output is treated as an annotation proposal. Human review can change the state,

revise the evidence note, or preserve uncertainty. The final reviewed JSONL, not the pre-label, is the source used for SFT and benchmark scoring. This distinction is essential because otherwise the benchmark would measure agreement with another model rather than agreement with reviewed visual facts.

4.4 Dataset Curation

4.4.1 Dataset Overview

Six reviewed datasets support the visual fact work across two ontology generations. Table 4.1 lists their roles and tracked counts.

Table 4.1: Dataset inventory used for visual fact extraction.

Run	Dataset	Samples	Facts	Lang.	Role
Run-001	v1 SFT	180	8	en/zh	v1 training source
Run-002	v1 holdout	50	8	en	v1 holdout
Run-002	v2 newfacts holdout	50	13	en	v2 holdout
Run-003	v2 SFT	220	13	en/zh	v2 training source
Run-004	v2 random holdout	100	13	en	v2 holdout
Run-005	composition rebalance	122	13	en/zh	v2 training supplement

Run-003 and Run-005 contain samples whose free-text summary field is blank (82 and 25 samples respectively). These samples remain valid for the visual fact task because every sample still contains a complete fact array. The supervised target used by the runtime and benchmark is the structured fact state, not the optional summary.

4.4.2 Dataset Quality Checks

The dataset pipeline applies several checks before a reviewed export is used for training or benchmarking. First, every sample must include the expected fact identifiers exactly once. Missing or duplicated fact IDs would make it unclear whether a model error is visual or structural. Second, every fact state must be one of the allowed states. Third, the image path must resolve to the composite panel image used during review. Fourth, training and holdout sets are checked for exact image overlap before the holdout metrics are interpreted as independent evidence.

The quality checks do not guarantee that every label is perfect. They guarantee that the data artifacts have the structure needed for a meaningful benchmark. This is a

different but important claim. In small-data adaptation, a few ambiguous labels can have a visible effect, especially for rare positive facts. The thesis therefore treats error analysis as part of the data workflow. If a holdout error reveals that a fact definition is too close to a procedural conclusion, the correct response is not simply to collect more data; it may be to revise the ontology.

Run-005 is an example of data-centric correction. The original Run-003 training source contained useful coverage but under-represented several rare states. The composition-rebalance supplement intentionally increases exposure to those states. This does not make the training set statistically representative of all possible cockpit states, and it is not presented that way. Its role is practical: expose the adapter to positive examples that matter for downstream procedure evidence and then test behaviour on independent holdouts.

4.4.3 Ontology Evolution: 8-Fact v1 to 13-Fact v2

The first ontology contained eight facts over the composite cockpit panel. It established that a small reviewed dataset could improve structured cockpit fact extraction, but it also exposed a representation problem: some targets mixed visual observation with procedural interpretation. The clearest case was `ins_go`, which asked a single-frame VLM to infer an INS completion state rather than report directly visible evidence.

Figure 4.2 illustrates the methodological correction. Compound procedural targets were decomposed into locally observable visual atoms, leaving temporal and procedural interpretation to the tutor runtime.

The current 13-fact ontology follows three design rules:

1. **Decompose compound targets.** The `ins_go` target is replaced by separate visual atoms for GRND alignment text and INS OK text. The FCS-MC sequence is represented by page, intermediate-result, in-test, and final-GO facts.

2. **Separate region from semantics.** Fact names describe the visible state rather than encoding the display region in the identifier. Region constraints are part of the ontology metadata.

3. **Keep facts locally observable.** Each fact should be decidable from the composite image without requiring procedure history. The procedure engine combines facts with telemetry, step bindings, and timing information later.

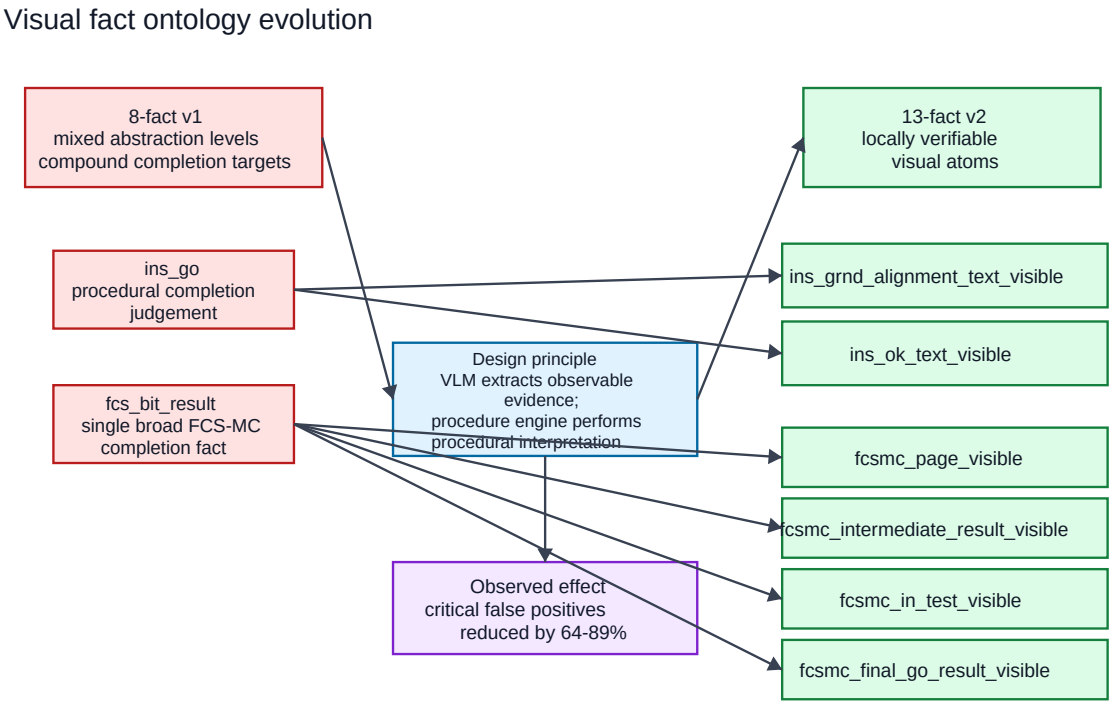


Figure 4.2: Ontology evolution from 8-fact v1 to 13-fact v2. Compound targets such as `ins_go` and broad FCS-MC completion facts were decomposed into locally verifiable visual atoms; the procedure engine then combines those observations with telemetry, phase, and timing evidence.

This ontology discipline is central to the thesis. It defines the boundary between perception and procedural reasoning: the VLM reports visual evidence, while the runtime decides whether that evidence satisfies a procedural step.

4.4.4 Training Recipe: Run-003 + Run-005×2

The primary v2 training recipe combines Run-003 with a repeated Run-005 composition-rebalance supplement. Run-003 contributes 220 reviewed images, exported bilingually as 440 SFT rows. Run-005 contributes 122 reviewed images, exported bilingually as 244 rows; this supplement is included twice, yielding 488 rows. The final recipe therefore contains **928 SFT rows**.

Run-005 targets under-represented positive states from Run-003, including INS OK text, FCS page X marks, and FCS-MC sub-state facts. The repeated supplement is a deliberate composition rebalance strategy, not an independent validation set. For that reason, generalisation claims are based on external holdouts rather than on an internal same-pool evaluation split.

4.4.5 Holdout Independence

Two 13-fact v2 holdouts are used for the main evaluation. Run-002 newfacts contains 50 images from an earlier capture session re-labelled under the 13-fact ontology. Run-004 random contains 100 randomly sampled cockpit-state captures. Local overlap checks report zero exact overlap between either holdout and the Run-003 + Run-005 training union.

The earlier Run-001 evaluation line is treated differently. Because it overlaps with the training data pool, it is useful as a contaminated development benchmark for checking schema learning and historical ontology behaviour, but it is not used as independent evidence for v2 generalisation.

4.5 LoRA Fine-Tuning Configuration

4.5.1 Backbone Models and Comparison Strategy

The primary adaptation line uses `Qwen/Qwen3.5-9B-Base` with the Run-003 + Run-005×2 recipe. Two supplementary backbones are included to test whether the same data recipe remains meaningful under scale, version, and family changes: `Qwen/Qwen3.6-27B`

and `google/gemma-4-31B`. These comparisons are methodological controls rather than separate thesis claims. Chapter 6 reports the numerical results.

The comparison holds the main factors constant wherever possible: ontology, training recipe, benchmark prompt, holdout sets, generation temperature, output schema, and metric computation. Backbone-specific differences remain and are therefore reported explicitly. Qwen-family runs apply LoRA to vision and language modules; the Gemma run uses an all-linear LoRA configuration with vision layers not separately fine-tuned. Full settings appear in Appendix 9, Table 9.1.

4.5.2 Training Stack and Hyperparameters

All backbone runs use 4-bit loading, LoRA rank $r = 16$, $\alpha = 16$, dropout 0.0, four epochs, learning rate 2×10^{-4}, effective batch size 4, maximum sequence length 4096, seed 3407, warmup ratio 0.1, and weight decay 0.01. Adapter weights are saved separately from the base model.

The epoch count and LoRA rank are chosen for small-data domain adaptation. With 928 SFT rows, the adapter must learn both the fixed JSON schema and the visual boundaries of cockpit facts. A very small number of epochs may not teach those boundaries reliably, while excessive training would risk memorising the capture sessions. Rank 16 provides moderate adaptation capacity without turning the adapter into a high-capacity task-specific model.

4.5.3 Bilingual SFT and Composition Rebalance

The bilingual SFT strategy creates two instruction variants per reviewed image: one English and one Chinese. The image and fact labels remain identical. This increases instruction-format coverage without requiring additional image annotation. It does not replace the need for new cockpit states or hard negative examples.

Composition rebalance is handled through Run-005. Because rare positive facts are especially important for downstream procedure reasoning, the training recipe increases their exposure by adding targeted captures and repeating the Run-005 bilingual rows. The method therefore separates two questions: whether the adapter can learn the ontology from the rebalanced training recipe, and whether the learned extractor generalises to independent holdouts.

4.5.4 Training Target Construction

The SFT target is intentionally smaller than the runtime observation object. It contains the short summary field and the array of fact objects with `fact_id`, `state`, and `evidence_note`. It excludes frame identifiers, file paths, confidence values, sticky/expiry metadata, and procedure suggestions. Those excluded fields are runtime management metadata, not visual labels. Including them would teach the model to emit data it cannot observe from the image and would make the output harder to validate.

The target also avoids asking for explanatory prose beyond short evidence notes. Long free-form explanations would make the SFT objective less focused and introduce additional evaluation ambiguity. The downstream language model can later turn facts into user-facing tutoring language. The VLM adapter is trained to supply the visual facts that such language may reference.

This separation keeps the training recipe aligned with the system architecture. The VLM is not rewarded for sounding helpful, predicting the next step, or explaining the procedure. It is rewarded for returning a complete, parseable, ontology-aligned fact vector. That is why benchmark gains in this thesis are interpreted as perception-layer gains, not as evidence that the full tutor is pedagogically better.

4.6 Benchmark Protocol

4.6.1 Metrics

The benchmark compares model predictions against human-reviewed ground truth at both sample and fact level. The metrics are selected to reflect downstream runtime risk:

- `json_valid_rate`: proportion of samples with parseable JSON output.
- `schema_valid_rate`: proportion of samples whose JSON contains the required fact identifiers exactly once and uses only valid states.
- `fact_accuracy`: three-class accuracy over all facts and samples.
- `macro_f1`: macro-averaged F1 over the three states, averaged over facts.
- `seen_f1`: F1 for the positive `seen` class, averaged over facts. This matters because missed or hallucinated visible states change procedure evidence.

- `sample_exact_match`: proportion of samples where all 13 facts match the reviewed labels.
- `critical_false_positive_count`: count of completion-critical facts predicted `seen` when the reviewed label is `not_seen` or `uncertain`.

Critical false positives are tracked separately because they are the most dangerous visual error for the tutor. A false positive on a completion-critical fact could make the procedure engine believe that a display state has been observed when it has not.

4.6.2 Base vs. LoRA Comparison

Base and LoRA configurations receive the same prompt, images, generation parameters, and output schema. Decoding uses temperature 0.0, maximum new tokens 1024, and seed 3407. Predictions are written to JSONL, parsed, normalised, and scored without post-hoc filtering. The comparison artifact reports LoRA-minus- base deltas, while Chapter 6 interprets those deltas.

4.6.3 Benchmark Categories

Benchmark runs are classified by evidence status. Contaminated development sets come from the same data pool as training and are used only for regression and historical interpretation. Held-out new-session benchmarks use independent captures with zero exact training overlap. This distinction determines the strength of the claim that can be made from a metric.

4.6.4 Benchmark Artifacts

Each benchmark run produces a standard directory with metrics for each model, per-sample predictions, error JSONL files, per-fact score CSVs, comparison summaries, human-readable reports, and charts. These artifacts form the evidence chain for the technical results reported in Chapter 6.

4.6.5 Error Review Protocol

The benchmark is followed by qualitative error review. The review focuses first on critical false positives, because those are the errors most likely to create unsafe evidence for a procedure gate. It then inspects false negatives for vision-priority facts, because

repeated false negatives can make the tutor ask for unnecessary checks or fail to recognise progress. Finally, it inspects schema and exact-match failures to identify whether the remaining problem is formatting, visual ambiguity, or fact-definition mismatch.

This review is deliberately tied to the runtime use case. A generic ML report might stop after accuracy and F1. For a tutor, the relevant question is what a specific error would do downstream. A hallucinated FCS-MC final GO result is more serious than a conservative `not_seen` prediction for a page that can be rechecked. A schema-invalid output is less visually interesting but more important operationally because it triggers fallback. The benchmark protocol therefore connects numerical metrics to the validation and degradation paths described in Chapter 3.

4.7 Human-Subject Pilot Methodology

A small formative VR pilot study was conducted to gather descriptive evidence about the tutor in use during the F/A-18C cold-start task. The pilot was not a powered controlled experiment. Its purpose was to exercise the runtime in a realistic VR setting, inspect interaction logs, and identify procedural error patterns for later study design.

4.7.1 Participants and Conditions

Nine participants (P01–P09) completed one trial each. The study used a between-subjects design with non-random assignment based on scheduling availability:

With tutor (n=4). P04, P05, P08, and P09 received overlay highlights, structured help text, telemetry-grounded step inference, and VLM visual fact support where applicable.

Without tutor (n=5). P01, P02, P03, P06, and P07 received no tutor assistance. Telemetry was still recorded for offline step inference and error coding.

Participant experience profiles are reported in Appendix 9, Table 9.5. Because assignment was not randomised and the sample is small, condition comparisons are descriptive only.

4.7.2 Task and Apparatus

All trials used the 33-step F/A-18C cold-start procedure in DCS World on a Meta Quest 3 via Quest Link. The composite cockpit panel was mirrored for experimenter

observation. Tutor-condition trials used the live runtime with remote model inference through an OpenAI-compatible endpoint. A trial ended when the participant completed the procedure or voluntarily stopped after being unable to progress.

4.7.3 Data Sources and Coding

The export pipeline produced per-trial summaries, per-step coding files, help-cycle logs for tutor trials, action timelines, quality-gate metadata, and session summaries. Post-trial forms recorded participant experience, confidence before the trial, raw NASA-TLX workload dimensions [11], condition-specific tutor ratings where applicable, general usability/user ratings, and free-text comments.

Step errors were coded with five categories: omission (OM), commission (CO), order error (OR), prompting/assistance error (PA), and state violation (SV). The automated coding output was reviewed manually using telemetry, help-cycle logs, and action timelines; the human-coded columns are the authoritative source for reported pilot results.

4.7.4 Questionnaire and Comment Instruments

The post-trial form records raw NASA-TLX workload dimensions, not weighted NASA-TLX. The thesis reports the raw average because the formative pilot was not designed to estimate a psychometric workload effect. The form also records pre-trial confidence, condition-specific tutor ratings where the tutor was available, general usability/user ratings, and short open comments. The rating values are kept on their recorded scale and are not normalised into a single composite effectiveness score.

The comments are used cautiously. They are not treated as coded qualitative themes with inter-rater reliability. Instead, they provide context for observed log and questionnaire patterns. For example, comments about small fixed checklist text help explain why a no-tutor participant could struggle in VR even when a procedure list was technically available. Comments about the AI identifying where the learner was stuck help explain why tutor-condition participants may have perceived the system as useful even when assistance- coupled errors remained.

This instrument design matches the pilot's formative status. The questionnaire and comments can reveal usability problems, trust signals, and cognitive frictions that should shape the next study. They cannot establish causal changes in workload, trust, or learning. That boundary is maintained in the results chapter by excluding P02's mismatched

questionnaire, reporting P08 with manual task coding, and treating P09 as an independent participant after the metadata correction.

4.7.5 Data-Quality Notes

Two trial-level caveats affect interpretation. First, P08's automatic summary reported incomplete progress, but manual telemetry review confirmed completion of all 33 steps. The manual coding is used for analysis, and the pre-review file is preserved for audit. Second, P02's questionnaire metadata contains an internal participant-id mismatch. P02's task metrics are retained, but P02 is excluded from questionnaire-derived workload, confidence, usability, trust, or comment summaries. P09 is treated as an independent participant after the export correction recorded in the trial metadata.

4.7.6 Analysis Approach

The pilot analysis reports per-participant outcomes and condition-level descriptive summaries. No null-hypothesis significance tests are performed. Observed durations are documented but not interpreted as completion-time comparisons because incomplete without-tutor trials ended at different stopping points. Questionnaire values are reported as raw descriptive scores only. The analysis focuses on completion status, manual error categories, tutor interaction patterns such as help requests, VLM calls, overlay actions, and fallback events, and short anonymised comments that clarify interaction frictions.

4.8 Summary

The methodology establishes the evidence chain used by the rest of the thesis. Telemetry and retrieval provide controlled runtime context. The visual fact pipeline produces a reviewed and benchmarked perception layer for RQ2. The 13-fact ontology separates perception from procedural reasoning. The LoRA protocol uses matched data and holdouts to evaluate small-data adaptation without relying on contaminated development sets. The pilot protocol provides formative VR evidence for RQ3 while keeping its claims descriptive. Together, these methods make the later results auditable without overstating what the available evidence can prove.

5 Reproducibility and Artifact Provenance

The preceding methods chapters define the evidence-grounded tutoring runtime, the visual-fact adaptation pipeline, and the formative Quest 3 pilot protocol. This chapter records how those methods are made reproducible in the repository. It deliberately avoids a module-by-module code tour. Implementation details are included only when they determine the provenance of a reported result, constrain a foundation-model output, or explain why a later claim can be audited from local artifacts.

The chapter therefore has a narrow role in the thesis structure. It connects the conceptual method to files, logs, configuration boundaries, and exported analysis tables. Chapter 6 then uses those artifacts as evidence for RQ1, RQ2, and RQ3.

5.1 Repository-Level Boundaries

The research artifact is organised around a separation between procedural knowledge, runtime logic, external systems, and exported evidence. The procedure pack stores the F/A-18C Lot 20 cold-start steps, gate definitions, telemetry bindings, UI targets, and visual-fact bindings. The Python runtime loads those declarations and produces evidence packets, model prompts, harness decisions, overlay actions, and event logs. DCS World, DCS-BIOS, the Quest 3 display path, model servers, and screenshot capture are treated as external adapters.

This boundary is important for reproducibility. A result in the thesis should be traceable to one of three artifact classes:

- **Declarative configuration:** procedure steps, telemetry maps, visual-fact ontologies, UI targets, source-policy rules, and replay-suite definitions.
- **Generated evidence:** JSONL event logs, trial summaries, step coding, help-cycle tables, benchmark predictions, benchmark metrics, and quality-gate reports.

- **Human-reviewed evidence:** reviewed visual labels, manual step coding overrides, questionnaire records after exclusion rules, and anonymised free-text comments.

The thesis does not treat generated text from the model as an unverified research observation. Model output becomes evidence only after it has passed a schema, mapping, harness, benchmark, or manual-review step appropriate to the claim being made.

5.1.1 Artifact Granularity

The repository stores artifacts at different granularities because the research questions require different forms of inspection. RQ1 needs help-cycle granularity: the relevant unit is a learner request, the evidence available at that moment, the model response, the validation decision, and the overlay result. RQ2 needs sample and fact granularity: the relevant unit is an image, its reviewed fact vector, the model prediction, and the per-fact confusion entry. RQ3 needs participant and step granularity: the relevant unit is a trial, a procedure step, a coded error, a questionnaire row, or a comment.

Keeping these granularities separate prevents evidence transfer. A high VLM accuracy number cannot be used to justify a participant outcome unless the live trial actually invoked the vision path and logged the resulting facts. A participant completion row cannot be used to claim that the model was always correct unless the help-cycle and validation logs support that interpretation. A questionnaire score cannot be used as task performance evidence. The thesis therefore reports each artifact class in the section where it has the right claim strength.

This granularity also helps future replication. A researcher who wants to check the VLM result can inspect benchmark directories without replaying a headset session. A researcher who wants to check a tutor-condition pilot claim can inspect trial summaries and step coding without retraining the VLM. A developer who wants to debug an overlay mistake can inspect one help cycle without re-running all benchmarks. This is the practical value of artifact provenance: it makes the research object divisible into auditable units.

5.2 Runtime Evidence Provenance

RQ1 depends on whether help-cycle decisions can be reconstructed. The runtime therefore writes append-only JSONL events and analysis exports rather than only rendering

transient headset guidance. A help cycle records its trigger context, telemetry snapshot, recent deltas, active step, gate diagnostics, retrieved snippets, visual-fact snapshot when available, prompt-construction metadata, model-response status, mapping and harness results, overlay execution status, and fallback or repair metadata.

The same event format supports live sessions and replay. Replay can feed a recorded DCS-BIOS stream through the telemetry map, procedure gates, source policy, and harness contracts used by the live runtime. This is the practical meaning of an auditable tutor in the thesis: a later reader can inspect why a help cycle was accepted, repaired, downgraded, or rejected without relying on the model's natural-language explanation alone.

The procedure pack also constrains the response schema at runtime. Step identifiers, overlay targets, and admissible evidence-reference types are not free-form strings invented by the LLM. They are derived from the active pack, UI map, source policy, and evidence packet. This provenance is relevant to RQ1 because it turns a model response into a checked proposal rather than an unbounded command.

5.3 Visual-Fact Training and Benchmark Provenance

RQ2 depends on benchmark numbers, not on informal inspection of model output. Every visual-fact metric reported in Chapter 6 is copied from local benchmark artifacts such as `metrics_base.json`, `metrics_lora.json`, `benchmark_summary.json`, and per-fact score files. The selected main figures in the results chapter are copied from the same benchmark-report directories.

The VLM evidence chain has five stages:

1. DCS World cockpit states are captured as fixed composite images containing the left Digital Display Indicator, the Advanced Multipurpose Color Display, and the right Digital Display Indicator.

2. Images are pre-labelled and then corrected during human review. The reviewed JSONL export is treated as ground truth for training and scoring.

3. Reviewed labels are converted into bilingual supervised fine-tuning rows. The English and Chinese rows share the same image and fact labels.

4. LoRA adapters are trained for the evaluated backbones with the Run-003 + Run-005×2 recipe.

5. Base and adapted models are benchmarked on independent holdouts with the same prompt, schema, generation settings, and metric computation.

This chain preserves the key machine-learning boundary of the thesis. The VLM is evaluated as a structured perception component. It predicts visual facts with states `seen`, `not_seen`, and `uncertain`; it does not decide procedure completion and does not directly choose tutor actions. Critical false positives are therefore reported separately because they are the visual error most likely to corrupt downstream evidence.

5.3.1 Benchmark Metric Traceability

The benchmark directories preserve more than the summary values used in the main tables. They include per-sample predictions, parsed outputs, error files, fact-level score tables, and comparison reports. This matters because a single metric can hide different causes. A drop in exact match may come from one systematic fact error, from many rare formatting errors, or from ambiguous labels. A critical false positive count can be inspected by fact to see whether the remaining risk is concentrated on INS text, FCS-MC final results, or another completion-like cue.

The thesis uses rounded values in the PDF, but the JSON artifacts retain the exact values. During writing, the reported Qwen3.5, Qwen3.6, and Gemma metrics were checked against `metrics_base.json`, `metrics_lora.json`, and `benchmark_summary.json`. This is why the results chapter can compare base and LoRA lines without relying on informal model-card summaries. The same traceability rule applies to selected figures: the main-text VLM charts are copied from benchmark-report assets, and the source directory remains part of the evidence map.

Metric traceability does not solve all validity problems. The holdouts are independent of the training union by exact-overlap checks, but they remain within the same simulator, cockpit layout, and procedure family. The benchmark therefore supports within-domain adaptation claims. It does not establish that the adapter will generalise to other aircraft, other cockpit layouts, other rendering conditions, or other simulator procedures without new reviewed data and holdout evaluation.

5.4 Pilot Export Provenance

RQ3 depends on exported pilot artifacts. Each participant trial is represented by a trial directory containing the raw event stream, trial summary, step coding, action timeline,

help-cycle table, session metadata, and quality-gate report. Tutor-condition trials additionally contain help-cycle and VLM-call counts. Post-trial CSV forms provide participant experience, confidence, NASA-TLX dimensions, tutor-specific ratings where applicable, general usability/user ratings, and free-text comments.

The reported pilot analysis applies three provenance rules. First, manual step coding is authoritative when it conflicts with automatic reconstruction. This rule is used for P08, whose automatic summary reported incomplete progress while manual telemetry review confirmed all 33 steps. Second, P02's task-log metrics are retained, but P02's questionnaire-derived values are excluded because the questionnaire metadata contains an internal participant-id mismatch. Third, P09 is analysed as an independent participant after the participant-correction note recorded in the metadata.

These rules keep the pilot claim bounded. The pilot can describe observed completion, errors, workload ratings, usability ratings, comments, and tutor-interaction counts in this small formative sample. It cannot support statistical significance, causal learning effects, retention, transfer, or completion-time claims.

5.4.1 Manual Review as a Provenance Step

Manual review appears in two places in the artifact chain. The first is visual label review for the VLM datasets. The second is pilot step coding. In both cases, review is not a hidden correction performed after results are known; it is a documented provenance step that determines which artifact is authoritative for the reported claim. Reviewed JSONL labels are authoritative for VLM training and scoring. Human-coded step columns are authoritative for pilot completion and error counts when automatic reconstruction disagrees.

P08 illustrates why this rule is necessary. The automatic trial summary reported incomplete progress because passive reconstruction did not capture all completion evidence for the relevant steps. Manual telemetry review confirmed 33/33 steps, so the thesis reports the manual coding and preserves the automatic summary as a data-quality note. This is not an attempt to hide automation failure. It is a useful result: automatic exports are valuable, but formative pilot data still require explicit review when visual, temporal, or passive telemetry evidence is incomplete.

P02 illustrates a different provenance problem. The task log and step coding remain usable, but the questionnaire metadata contains an internal participant- identifier mismatch. The thesis therefore keeps P02 for task metrics and excludes P02 from questionnaire-derived summaries. This avoids discarding a whole trial unnecessarily

while preventing mismatched subjective data from entering workload, confidence, usability, trust, or comment summaries.

5.5 Figure and Table Provenance

The main text uses simplified figures for argument clarity and selected benchmark figures for empirical support. Architecture and pipeline diagrams are derived from repository-level design artifacts and rendered into the `paper/figures/` directory. Benchmark charts are copied from the VLM training-report assets so that the PDF does not depend on external paths at compile time.

Tables in Chapter 6 follow a stricter rule: numerical model metrics are taken from benchmark JSON artifacts, and pilot metrics are taken from exported trial summaries, manually reviewed step-coding tables, and questionnaire CSV files after the exclusion rules above. When the main text uses rounded values, the underlying artifact retains the exact values.

5.5.1 Audit Trail for Claims

The thesis uses a simple claim-audit logic. A claim about architecture must be supported by a method description, configuration boundary, or runtime log. A claim about VLM reliability must be supported by a benchmark artifact. A claim about participant behaviour must be supported by trial exports and manual step coding. A claim about participant perception must be supported by questionnaire or comment artifacts after exclusion rules. Claims that lack the right artifact type are either removed or explicitly marked as future work.

This audit trail is especially important for negative boundaries. The thesis states several things it does not prove: learning gains, retention, transfer, statistical significance, workload reduction, faster completion, and broad cross-domain generalisation. These limits are not rhetorical hedges. They follow from the artifact chain. The current data include component benchmarks, replay/fixture evidence, and a small formative pilot. They do not include a randomised powered study, retention test, transfer task, or multi-domain benchmark. The claim boundary is therefore a property of the evidence, not only of cautious writing.

The audit trail also makes the thesis extensible. If a later study adds a randomised controlled pilot, the new evidence can be added as a new artifact class rather than

rewriting the whole runtime argument. If a later model improves the VLM benchmark, the same benchmark schema can compare it against the current Qwen and Gemma lines. If a new procedure pack is added, its evidence-source mapping can be audited against the same design principles.

5.6 Reproducibility Limits

The repository supports local inspection and LaTeX compilation of the thesis, but complete reruns of every experiment require external state: DCS World, an F/A-18C module installation, DCS-BIOS integration, a Quest 3 setup, model server access, and GPU resources for VLM fine-tuning and benchmarking. The thesis therefore distinguishes *artifact reproducibility* from *full experimental rerun reproducibility*. The former is the claim made here: the reported results can be traced to stored artifacts and checked for internal consistency. The latter would require recreating the simulator, hardware, model, and participant conditions.

This distinction also applies to the original thesis proposal. The proposal described a broader VR tutoring study; the completed thesis reports a narrower but more auditable artifact. The evidence available now supports an evidence-grounded ML tutoring runtime, a small-data VLM adaptation result, and a formative Quest 3 pilot. A larger controlled human-subject study remains future work.

6 Results

This chapter reports the results in the same order as the research questions: RQ1 for the evidence-grounded tutoring runtime, RQ2 for VLM adaptation, and RQ3 for the formative Quest 3 pilot. The three evidence lines have different claim strengths. Replay, harness, and export artifacts support a technical auditability claim for RQ1. Holdout benchmarks support a component-level machine-learning claim for RQ2. The pilot supports descriptive observations about live use, task completion, errors, workload ratings, usability/trust ratings, and comments for RQ3.

Figure 6.1 summarises the evidence flow. The important point is that the three streams are not interchangeable. A VLM benchmark is not a learning outcome; a replay fixture is not a participant result; and a formative pilot is not a powered controlled experiment.

6.1 RQ1: Evidence-Grounded Tutoring Runtime

RQ1 asks how an evidence-grounded foundation-model tutoring runtime can integrate telemetry, retrieval, visual facts, and validation. The result is an operational runtime whose help-cycle decisions can be reconstructed from stored artifacts. The evidence is technical rather than pedagogical: configuration files define what evidence is admissible, replay and fixtures exercise known failure modes, and live pilot exports show that the same logging path ran during headset use.

6.1.1 Runtime Readiness Summary

Table 6.1 summarises the RQ1 readiness evidence. It does not claim that every future help response will be correct. It shows that the current runtime places explicit checks between model generation and learner guidance.

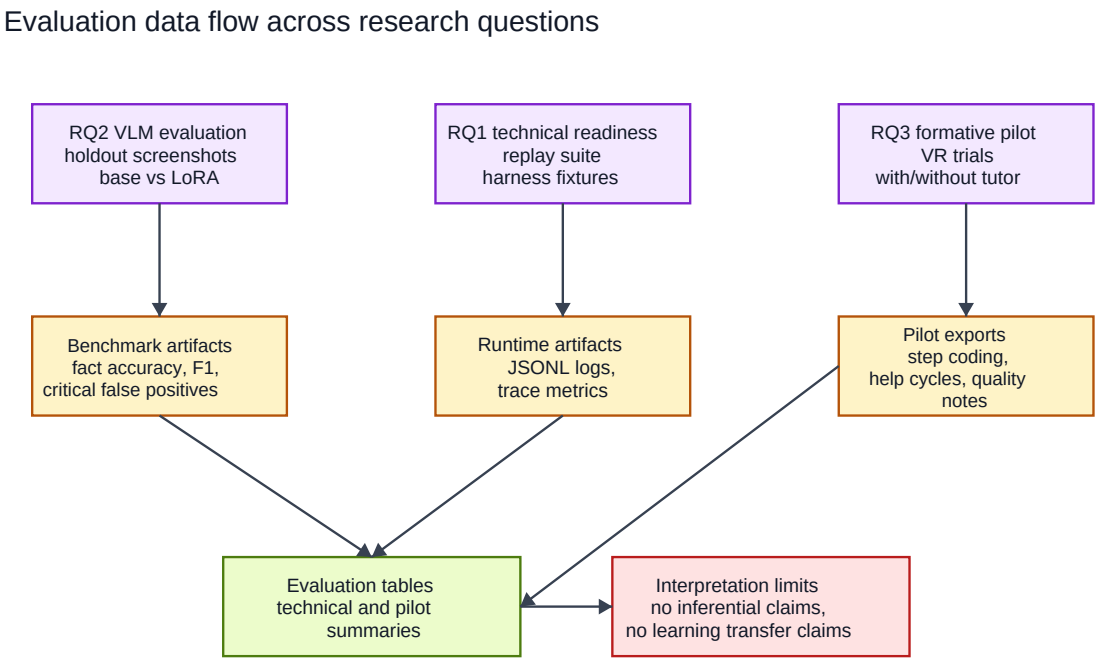


Figure 6.1: Evaluation and data flow across research questions. Replay and harness artifacts support RQ1, VLM benchmark artifacts support RQ2, and Quest 3 pilot exports support RQ3 descriptive findings.

Table 6.1: RQ1 runtime-readiness evidence.

Runtime element	**Observed artifact**	**Resulting claim boundary**
Procedure pack	33-step F/A-18C Lot 20 cold-start pack derived from manual/training material	Step order, gate rules, UI targets, and source policy are explicit rather than invented by the model.
Telemetry grounding	DCS-BIOS variables, normalised runtime variables, missing-source tracking	Observed values and absent sources remain distinguishable in the help context.
Retrieval grounding	Source-controlled BM25 snippets	Retrieved text is scoped to allowed procedure/document sources.
Visual fact channel	Vision-priority bindings for S08, S15, S18, and S19	Display-page facts can enter the evidence packet without giving the VLM procedural authority.
Schema and harness	Pack-derived response schema, UI allowlists, evidence-reference checks	Model proposals are validated, repaired, downgraded, or rejected before overlay execution.
Replay and fixtures	Five replay cases and 20 live-fixture regression entries	Known grounding failures can be reproduced and checked outside a live headset session.
Pilot export	Nine participant exports with summaries, step coding, help cycles, and quality gates	The live runtime produced audit-ready artifacts during the formative Quest 3 pilot.

6.1.2 Replay and Harness Results

The replay suite contains five cases covering early electrical power, engine and generator state, FCS reset/BIT interaction, and INS-mode evidence. Cases marked as vision available include sidecar frame files so that vision-dependent logic can be exercised without a live simulator. These cases confirm that recorded DCS-BIOS streams can be passed through the same procedure pack, telemetry map, source policy, UI map, and validation path used during live tutoring.

The live-fixture regression matrix contains 20 entries. The entries cover five readiness risks: advancement when a step is already complete, wrong-target prevention and repair, stale VLM evidence leaking into non-visual steps, waiting behaviour for moving or settling controls, and preservation of interaction semantics such as mouse-wheel controls. This fixture evidence supports the RQ1 auditability claim: the system has executable checks for specific evidence-grounding failures. It does not prove educational effectiveness.

6.1.3 Live Export Results

All nine participant trials produced trial summaries, step coding, action timelines, raw-event copies, session metadata, and quality-gate reports. The four tutor-condition trials also produced help-cycle tables. No participant export had a blocking quality-gate failure. One trial, P04, retained metadata warnings for missing or incomplete linkage fields; those warnings limit traceability for those fields but do not change the manually reviewed task metrics reported below.

Across the four tutor-condition trials, the runtime logged 77 help cycles, 23 VLM calls, 82 overlay executions, zero overlay rejections, and 21 fallback or repair events. The zero overlay-rejection count should be interpreted carefully: it means final overlay actions passed the configured allowlist and evidence checks, not that model proposals were always directly correct. The 21 repair/fallback events show that the validation layer was active during live use.

6.1.4 Example Evidence Path

The RQ1 result can be read as an evidence path rather than as a single system status. A typical vision-priority help cycle begins with telemetry-derived procedure context: the runtime identifies the current candidate step, evaluates preconditions and completion gates, and records unavailable sources separately from false values. If the active step

requires display evidence, the runtime requests or selects a composite-panel frame and obtains structured visual facts. Retrieval then adds source-controlled procedural context. Only after these steps does the LLM receive a prompt.

The model response is not immediately displayed. The JSON object is parsed, checked against the pack-derived schema, mapped to allowed response fields, and validated against UI target bindings and evidence references. If the model proposes a target that is too broad, the harness can repair it to a more specific target. If evidence is missing, the overlay can be suppressed or the response can degrade to text-only guidance. The event log preserves the telemetry snapshot, visual fact snapshot, prompt metadata, raw model output, mapping decision, harness decision, and overlay result.

This example matters because it clarifies what RQ1 does and does not show. The runtime result is not "the model gave good answers". The result is that a help cycle can be decomposed into evidence production, model generation, validation, repair, and export. That decomposition is the technical foundation needed before a later controlled study can ask whether the tutor improves learning, trust, workload, or transfer.

6.2 RQ2: VLM Adaptation for Cockpit Visual Facts

RQ2 asks whether a small-data LoRA-adapted VLM can improve reliable cockpit visual fact extraction. The primary result line is Qwen3.5-9B trained with the Run-003 + Run-005×2 recipe. Qwen3.6-27B is reported as a larger deployable production candidate under the same recipe, and Gemma-4-31B is reported as a comparison backbone. All metrics in this section are copied from benchmark JSON artifacts and computed on independent 13-fact holdouts.

6.2.1 Dataset and Training Summary

The v2 training recipe combines Run-003 (220 reviewed images) with Run-005 (122 reviewed images) repeated twice as a composition-rebalance supplement. Bilingual export yields 928 supervised fine-tuning rows. The two main holdouts are Run-002 newfacts (50 images) and Run-004 random (100 images), both scored under the 13-fact ontology with zero exact overlap against the training union.

Table 6.2: Visual-fact dataset inventory for the reported VLM results.

Source	Run	Facts	Images	Role
v1 training source	Run-001	8	180	Historical 8-fact training source
v1 holdout	Run-002	8	50	Historical 8-fact holdout
v2 newfacts holdout	Run-002	13	50	Main holdout
v2 training source	Run-003	13	220	Main 13-fact training source
v2 composition rebalance	Run-005	13	122	Rebalanced supplement, used twice
v2 random holdout	Run-004	13	100	Main holdout

6.2.2 Primary Qwen3.5 Result

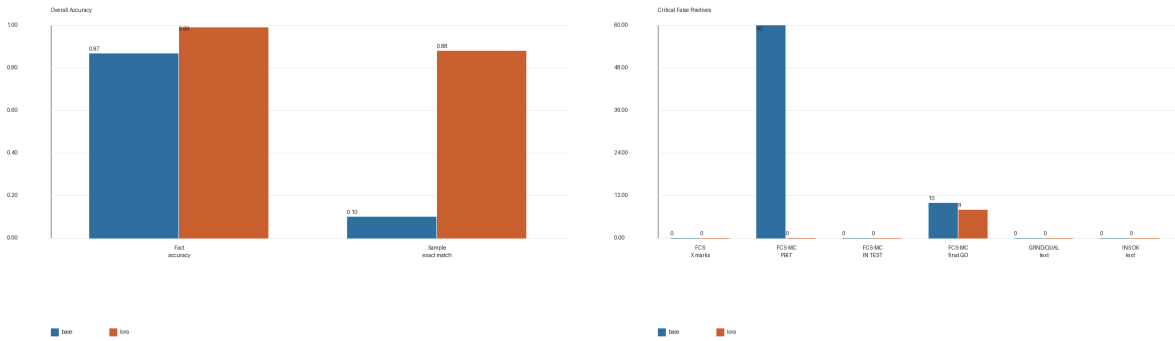
Table 6.3 reports the main Qwen3.5 base-versus-LoRA result. The adapted model improves JSON/schema reliability on Run-004, raises fact accuracy to approximately 0.99 on both holdouts, and reduces critical false positives from 11 to 4 on Run-002 and from 70 to 8 on Run-004.

Table 6.3: Primary 13-fact result: Qwen3.5-9B base vs. LoRA on two holdout sets.

	Run-002 newfacts (50)			Run-004 random (100)		
Metric	**Base**	**LoRA**	**Δ**	**Base**	**LoRA**	**Δ**
JSON valid	1.0000	1.0000	0.0000	0.9600	1.0000	+0.0400
Schema valid	1.0000	1.0000	0.0000	0.9600	1.0000	+0.0400
Fact accuracy	0.8677	**0.9908**	+0.1231	0.8038	**0.9931**	+0.1893
Macro F_1	0.4513	**0.6515**	+0.2002	0.4393	**0.6100**	+0.1707
Seen F_1	0.5022	**0.9648**	+0.4626	0.5659	**0.9159**	+0.3500
Exact match	0.1000	**0.8800**	+0.7800	0.0300	**0.9200**	+0.8900
Critical FP	11	**4**	−7	70	**8**	−62

Figure 6.2 adds the corresponding training-report figures selected for the main text. The first chart visualises the Run-002 accuracy improvement; the second highlights why critical false positives are tracked separately from aggregate accuracy on Run-004.

The practical interpretation is not that the VLM can run the procedure. The adapter makes the visual-fact channel much more usable as one evidence source. Completion-like facts remain the main residual risk and must still be checked downstream by the procedure engine, telemetry, temporal context, and harness constraints.



(a) Run-002 overall accuracy.

(b) Run-004 critical false positives.

Figure 6.2: Selected Qwen3.5 benchmark figures from the VLM training report.

6.2.3 Metric Interpretation for the Primary Result

The Qwen3.5 result is strongest when the metrics are interpreted together. JSON and schema validity show whether the model can participate in the runtime contract at all. On Run-004, the base model fails this contract for four samples, while the LoRA adapter returns valid and schema-valid objects for all samples. This matters because runtime rejection and fallback are triggered by format and schema failures before visual correctness is considered.

Fact accuracy shows the broad improvement, but it understates the importance of seen-state behaviour. The visual facts that matter most during tutoring are often positive cues: a page is visible, a status text has appeared, or a final GO result can be read. Seen F_1 therefore captures whether the adapter is better at recognising visible cues without hallucinating them too often. The increase from 0.5022 to 0.9648 on Run-002 and from 0.5659 to 0.9159 on Run-004 is the most direct evidence that the adapter learned the positive visual states needed by the runtime.

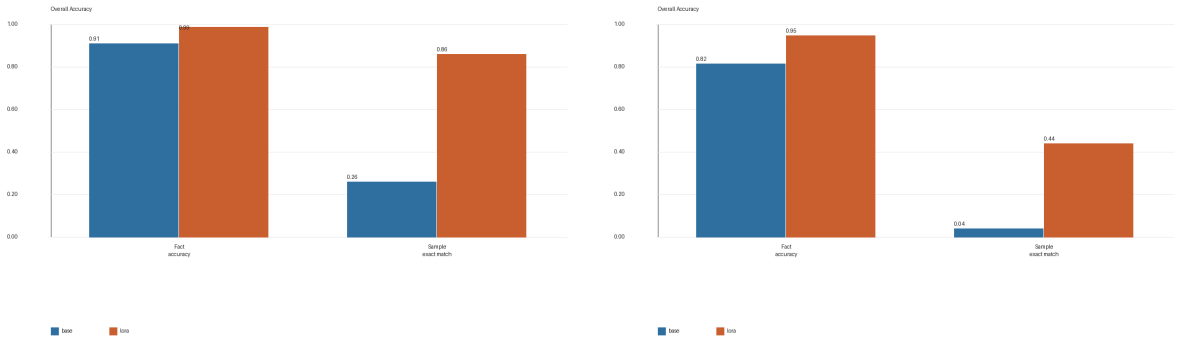
Sample exact match adds another view. A help cycle can carry a whole visual snapshot, so one wrong fact can still produce a misleading context. The Qwen3.5 adapter raises exact match from 0.10 to 0.88 on Run-002 and from 0.03 to 0.92 on Run-004. This suggests that the adapted model is not merely improving a few easy facts while leaving most snapshots inconsistent. The critical-false-positive reduction is the final safety-oriented check: the adapter reduces the errors most likely to make the downstream tutor overstate a completion cue.

6.2.4 Qwen3.6 and Gemma Comparison

Table 6.4 reports the matched supplementary backbone results. Qwen3.6-27B has a stronger base model than Qwen3.5 on these holdouts, especially on Run-004, and remains a deployable production candidate after LoRA adaptation. However, the Qwen3.6 LoRA does not surpass the Qwen3.5 LoRA on the primary metrics in this local benchmark. Gemma-4-31B improves under the same recipe, but its adapted exact-match and seen-F_1 values remain below the Qwen adapters, with a different critical-false-positive profile.

Table 6.4: Supplementary backbone comparison under the matched Run-003 + Run-005×2 recipe.

Backbone	Model	Run-002 newfacts				Run-004 random			
		Acc.	Seen F_1	Exact	Crit. FP	Acc.	Seen F_1	Exact	Crit. FP
Qwen3.5-9B	Base	0.8677	0.5022	0.1000	11	0.8038	0.5659	0.0300	70
Qwen3.5-9B	LoRA	**0.9908**	**0.9648**	**0.8800**	4	**0.9931**	**0.9159**	**0.9200**	8
Qwen3.6-27B	Base	0.9108	0.6712	0.2600	1	0.9769	0.8211	0.7000	7
Qwen3.6-27B	LoRA	0.9877	0.9479	0.8600	3	0.9892	0.9147	0.8600	7
Gemma-4-31B	Base	0.8169	0.5128	0.0400	16	0.8146	0.6332	0.1200	47
Gemma-4-31B	LoRA	0.9492	0.8123	0.4400	**0**	0.9715	0.7959	0.7400	13



(a) Qwen3.6 Run-002 overall accuracy. **(b)** Gemma Run-002 overall accuracy.

Figure 6.3: Selected supplementary backbone benchmark figures from the VLM training reports.

The comparison supports a conservative conclusion. The data-and-ontology recipe is not tied to a single evaluated backbone, but deployment choice is model-specific. Qwen3.5 remains the main thesis result because it gives the best adapted accuracy/exact-match combination in the current holdouts. Qwen3.6 is attractive as a production candidate because its base line is already stronger, while Gemma provides useful comparison evidence about a different error style.

6.2.5 Model-Specific Interpretation

The Qwen3.6 result is useful because it separates backbone strength from LoRA gain. Its base model is already much stronger than the Qwen3.5 base on the two holdouts, especially on Run-004 where it reaches 0.9769 fact accuracy and 0.70 exact match before adaptation. The LoRA adapter still improves seen F_1 and exact match, but the relative gain is smaller because the base line is already closer to the task. This is why Qwen3.6 is described as a deployable production candidate rather than as the primary thesis result: it is strong, but it does not change the main methodological conclusion that small-data adaptation plus a good ontology can make a VLM usable as a bounded perception component.

Gemma shows the opposite pattern. Its base line is weaker than the Qwen3.6 base and close to the Qwen3.5 base in broad accuracy, but the adapter reduces critical false positives to zero on Run-002. At the same time, its exact match and seen F_1 remain lower than the Qwen adapters. This suggests a more conservative or incomplete adapted behaviour on the local data: fewer critical hallucinations in one holdout, but less complete recognition of positive states. For a tutor, that trade-off is not automatically worse or better. A conservative model may be safer for some display cues, but it may also force more repeated checks or fallback guidance.

The matched comparison therefore supports a design conclusion rather than a leaderboard conclusion. Backbone choice should be made with downstream error costs in mind. A model with slightly lower aggregate accuracy may be acceptable if it avoids critical false positives for a safety-sensitive fact. A model with higher exact match may still need harness constraints if its residual errors cluster around completion cues. The thesis keeps Qwen3.5 as the main line because it has the best adapted balance in these benchmarks, while documenting Qwen3.6 and Gemma so that the model-choice boundary is visible.

6.2.6 Error Pattern

The base Qwen3.5 model's dominant failure mode is rare-state hallucination. In Run-004, the FCS-MC intermediate-result fact appears in only 7 of 100 samples, yet the base model predicts it as visible in many additional samples. LoRA adaptation removes most of this diffuse hallucination pattern. The remaining adapted errors are fewer and more concentrated around completion-like display signals, which are exactly the facts that the runtime treats with caution.

This pattern explains why the 13-fact ontology matters. Page-presence facts are easier to learn than procedural completion states. Decomposing compound targets into locally observable visual atoms gives the model a better supervised task and gives the runtime a safer evidence boundary.

The residual error pattern is also informative for future data collection. Additional random screenshots would not necessarily solve the hardest cases if they mostly repeat already easy page-presence states. The missing data are targeted hard negatives and temporally adjacent states: frames where a page is present but a final status is not yet visible, frames where intermediate and final BIT states look similar, and frames where display text is small or transient. These examples are more valuable than generic screenshots because they directly attack the remaining critical-false-positive risk.

The benchmark therefore acts as an ontology audit. When a fact is easy for the adapter and stable across holdouts, it is probably a good local visual proposition. When a fact remains error-prone after adaptation, the problem may be insufficient data, visual ambiguity, or a fact definition that still embeds too much procedural reasoning. The thesis treats these cases as feedback to the representation, not only as model shortcomings.

6.3 RQ3: Formative Quest 3 Pilot

RQ3 asks what descriptive evidence a formative, non-random Quest 3 pilot provides about task completion, procedural errors, workload, trust/usability, comments, and the boundary of a trustworthy AI collaborator. Nine participants completed one trial each: four with tutor assistance and five without tutor assistance. The pilot is reported descriptively only. It does not support statistical significance, learning-gain, retention, transfer, workload- reduction, or speed claims.

6.3.1 Participants and Task Outcomes

Table 6.5 summarises trial outcomes and interaction counts. P08 uses the manual coding override described in Section 4.7; P09 is treated as an independent participant. Observed duration is the interval from first to last recorded event and is not a completion-time comparison because without-tutor trials ended at different procedure points.

In the reviewed data, all four tutor-condition participants reached the final 33-step state, while none of the five without-tutor participants did. The most commonly missed

Table 6.5: Participant and trial overview.

P	Condition	Completed	Steps	Dur. (s)	Help	VLM	Ovrl.	Fallbk.
P01	without_tutor	no	23/33	2015	0	0	0	0
P02	without_tutor	no	27/33	739	0	0	0	0
P03	without_tutor	no	24/33	243	0	0	0	0
P06	without_tutor	no	29/33	269	0	0	0	0
P07	without_tutor	no	19/33	218	0	0	0	0
P04	with_tutor	yes	33/33	442	19	6	22	6
P05	with_tutor	yes	33/33	317	7	2	6	2
P08	with_tutor	yes*	33/33	332	7	5	10	4
P09	with_tutor	yes	33/33	1033	44	10	44	9

*P08: automatic summary reported incomplete progress; manual coding confirms 33/33 completed.

without-tutor steps were S18 (FCS-MC BIT page), S19 (FCS BIT final GO), and S27 (flap switch to AUTO), each missed by all five without-tutor participants. This pattern identifies where participants struggled in the formative pilot; it is not a causal estimate of tutor effect.

The participant profiles make the descriptive boundary especially important. The sample includes novices, intermediate users, and expert users, but the conditions are not balanced by prior DCS World or F/A-18C experience. P05 and P06 were expert participants in different conditions, while P01 and P09 were novices. P09 is treated as an independent participant after the export correction, but the pilot still cannot rule out experience, scheduling, or session-context effects. The correct interpretation is therefore not "the tutor caused completion". The correct interpretation is that the completed runtime could support four tutor-condition participants through the full procedure in this formative sample, while the no-tutor runs revealed recurring failure points that deserve controlled follow-up.

6.3.2 Procedural Error Pattern

Table 6.6 reports the manual error counts. The without- tutor trials are dominated by omissions, while the tutor trials shift toward commission and prompting/assistance errors. This shift is important for future design: assistance can keep a participant moving through the procedure while still introducing timing, interpretation, or over-guidance risks.

Without-tutor participants averaged 7.4 manually coded errors. Tutor-condition par-

Table 6.6: Manual procedural error counts. OM = omission, CO = commission, OR = order error, PA = prompting/assistance error, SV = state violation.

P	Condition	OM	CO	OR	PA	SV
P01	without_tutor	8	0	1	0	1
P02	without_tutor	0	0	0	1	0
P03	without_tutor	7	0	2	0	2
P06	without_tutor	2	0	0	2	0
P07	without_tutor	11	0	0	0	0
P04	with_tutor	0	3	0	3	1
P05	with_tutor	0	3	0	3	0
P08	with_tutor	0	0	0	0	0
P09	with_tutor	0	3	0	2	0

ticipants averaged 4.5 manually coded errors. These are descriptive statistics from a small non-random pilot. The more robust qualitative result is the error-type contrast: omissions dominate without tutor assistance, while assistance-coupled errors remain in the tutor condition.

The assistance-coupled errors are important because they show that a trustworthy assistant is not merely a completion machine. A participant can complete all steps while still receiving guidance that is mistimed, redundant, or too eager after a state has already been achieved. In the pilot, commission and prompting/assistance errors in the tutor condition point to these interaction risks. They motivate stricter confirmation logic, better detection of already-completed state, and clearer uncertainty handling in future versions.

6.3.3 Workload, Usability, Trust, and Comments

P02 is excluded from all questionnaire-derived summaries because the questionnaire metadata contains an internal participant-id mismatch. Table 6.7 reports the remaining raw questionnaire values. Scores are descriptive only. The tutor-specific rating is available only in the with-tutor condition; the general user/usability rating is available for both conditions.

The raw TLX values range from 13.33 to 55.83 without tutor and from 13.33 to 33.00 with tutor. The descriptive means are 34.12 and 23.66 respectively, but the sample is too small and unbalanced for a workload-reduction claim. The tutor-specific ratings in the with-tutor group are consistently above the midpoint of the scale, with a mean of 5.92. These scores support a formative interpretation: participants generally rated the

Table 6.7: Post-trial questionnaire summary after excluding P02. Ratings use the recorded 1–7 response scale where applicable.

P	Condition	Confidence	Raw TLX	Tutor rating	User rating
P01	without_tutor	3.5	55.83	n/a	4.20
P03	without_tutor	6.0	26.67	n/a	5.20
P06	without_tutor	7.0	13.33	n/a	5.60
P07	without_tutor	5.0	40.67	n/a	5.20
P04	with_tutor	7.0	26.33	6.00	6.20
P05	with_tutor	7.0	13.33	6.17	5.60
P08	with_tutor	5.0	33.00	5.67	5.00
P09	with_tutor	3.5	22.00	5.83	6.40
Mean without tutor		5.38	34.12	n/a	5.05
Mean with tutor		5.63	23.66	5.92	5.80

tutor positively, while the logs and error coding still show unresolved assistance risks.

The questionnaire values are useful because they complicate a purely log-based reading. P09 had many help requests and the longest tutor-condition trace, yet the recorded user rating was high and the comments described grounded feedback as helpful. P05, an expert participant, needed fewer help cycles and recorded a low TLX value, but still had assistance-coupled errors. These examples suggest that future evaluation should not reduce the tutor to one outcome variable. Completion, workload, perceived usefulness, trust, error type, and help-cycle repair frequency may move differently for novices and experts.

Table 6.8 summarises short anonymised comments. The comments are not coded as themes with inter-rater reliability; they are used to explain concrete interaction frictions observed in the pilot.

6.3.4 RQ3 Summary

The pilot provides three descriptive findings. First, the runtime could be used live in a Quest 3 setting and produced analyzable exports. Second, reviewed task logs show complete progression for all four tutor-condition participants and incomplete progression for all five without-tutor participants in this non-random sample. Third, the tutor changed the observed error pattern rather than eliminating all errors: omissions disappeared in the tutor condition, but commission and prompting/assistance errors remained. These findings motivate the future-work question of how AI can become a trustworthy collaborator in dynamic, high-load, multimodal environments rather than merely a fluent

Table 6.8: Short anonymised participant comments, paraphrased or lightly condensed for space.

Condition	Theme	Comment summary
Without tutor	Checklist legibility	A participant reported that kneeboard reading was difficult and that small fixed text was hard to use in VR.
Without tutor	Procedure mapping	A participant reported that some checklist steps did not match their cockpit experience clearly enough.
With tutor	Missed operations	A participant reported that the tutor helped fill in operations they had missed.
With tutor	Grounded feedback	A novice participant described step-by-step guidance that identified where they were stuck and did not advance when the action was wrong.
With tutor	Cognitive effort	A participant still needed to compare cockpit displays with AI instructions and found abbreviations such as FCS and BIT confusing.

source of instructions.

7 Discussion

Chapter 6 reported three evidence lines with different claim strengths: replay and harness readiness evidence for RQ1, visual-fact benchmarks for RQ2, and a formative VR pilot for RQ3. This chapter interprets those evidence lines together. The main claim is not that the tutor has already been proven to improve learning. Rather, the thesis shows how a procedural tutor can be made technically auditable by placing telemetry, procedural rules, retrieved sources, VLM-derived visual facts, and harness validation between the learner and the foundation-model response.

Section 7.1 explains how the implemented work differs from the original proposal. Section 7.2 interprets the technical evidence. Section 7.3 discusses the pilot findings within their formative boundary. Section 7.4 draws the main design lesson from the ontology evolution. Sections 7.5 and 7.6 state the limits of the thesis and the most direct next research steps.

7.1 Evolution from Proposal to Implemented System

The original thesis proposal [26] described a VR-first tutoring system for the Meta Quest 3, with gaze, hand tracking, voice input, a RAG+LLM backend, and a controlled three-condition human-subject study. The final thesis keeps the immersive procedural tutoring motivation, but the center of the work moved. The implemented contribution is best understood as an evidence-grounded tutoring runtime and visual-fact extraction method, with VR used for a formative pilot rather than for a controlled effectiveness claim.

This shift was not only an engineering convenience. The cold-start procedure made a prior problem visible: before a tutor can give useful procedural help, it must know which state it is allowed to trust. Some required evidence is exported as telemetry through DCS-BIOS. Some evidence is visible only on cockpit displays. Some evidence is outside the current telemetry or visual-fact contract, so the runtime must treat it as unavailable rather than silently turning it into a confident state claim. The thesis therefore had to

build the evidence layer first: procedure packs, gate inference, evidence packets, retrieval boundaries, VLM visual facts, schema-constrained model output, harness validation, overlay allowlists, and replayable logs.

The VLM work grew out of the same pressure. The proposal assumed that simulator state would be sufficient for procedural tracking. In practice, display-page content such as alignment and BIT states had to be observed visually. This led to the 13-fact ontology, the reviewed screenshot datasets, the LoRA adaptation pipeline, and the independent holdout benchmarks. These became a core contribution because they define how a foundation model can be used as a bounded perception component rather than as an unconstrained judge of procedure state.

The completed VR pilot partially returns to the original proposal's human-subject ambition. It shows that the evidence-grounded runtime can be exercised in headset with real participants and that the export pipeline can capture task logs, workload ratings, usability/trust ratings, and comments for later analysis. It does not, however, complete the controlled three-condition study proposed at the beginning of the project. With $n = 9$, non-random assignment, P02 excluded from questionnaire-derived summaries, and P08 requiring manual completion coding, the pilot is formative and descriptive.

7.2 Interpretation of Technical Evidence

The strongest technical result is the RQ2 visual-fact benchmark. The primary Qwen3.5-9B LoRA line reaches approximately 0.99 fact accuracy on both independent 13-fact holdouts, while reducing critical false positives sharply relative to the base model. Supplementary Qwen3.6-27B and Gemma-4-31B results show that the same data-and-ontology recipe is not tied to a single backbone, although the thesis does not claim broad model-family generality. The important interpretation is narrower and more useful: a small, reviewed, ontology-aligned dataset can make a VLM substantially more reliable for cockpit visual-fact extraction than the corresponding base model on local holdout data.

That result matters because the tutoring runtime treats VLM output as evidence, not as authority. A visual fact such as a visible page, label, or status text is one input to downstream step inference. The procedure engine still combines visual facts with telemetry, procedure order, temporal context, and pack-defined gates. This separation is what prevents the benchmark from becoming an unsupported pedagogical claim. The VLM result supports the perception layer of the tutoring system; it does not by itself prove that the tutor teaches better.

The RQ1 replay and harness evidence supports a different part of the thesis. The replay cases, live-fixture coverage, pack-derived schemas, response mapping, and overlay allowlists show that the implemented runtime can constrain model output before it reaches the learner. This is a technical readiness and auditability claim. It means that help cycles can be replayed, inspected, and checked against explicit evidence contracts. It does not mean that every future model response will be pedagogically optimal, nor that the runtime has been validated across simulators or procedures.

Taken together, the technical evidence supports the architecture's central premise: foundation models can be useful in procedural tutoring when their role is bounded by explicit evidence production and validation. The thesis does not ask the LLM to be the source of truth. It asks the LLM to generate a response inside a runtime where the observable state, procedural pack, retrieved sources, and harness define what the response may safely claim or highlight.

7.2.1 What the Technical Evidence Does Not Prove

The technical evidence does not show learning gains, retention, transfer, reduced workload, or superior training outcomes. The VLM benchmarks measure single-frame visual-fact extraction. The replay and harness tests measure whether evidence contracts and output constraints behave as expected. Both are necessary for a reliable tutor, but neither is sufficient for a human learning claim. A learner could receive accurate, grounded help and still fail to form a durable procedural understanding. Conversely, a learner might learn from a less instrumented system if it happens to fit their prior knowledge and strategy.

The evidence also does not yet isolate the value of each component. This thesis does not contain an ablation comparing telemetry-only tutoring, vision-augmented tutoring, different retrieval policies, or different harness strictness levels. The evaluation therefore supports an integrated artifact claim: the implemented system shows a technically auditable way to combine these elements. It does not rank every design choice against alternatives.

7.2.2 Data-Centric Interpretation of the VLM Result

The VLM result is partly a model result, but it is more importantly a data-and-representation result. The most visible improvement comes from LoRA adaptation, yet the adapter is only one part of the system that changed between the weak early

visual pipeline and the current result. The ontology was revised, the label contract was narrowed, the training distribution was rebalanced, and the benchmark separated independent holdouts from contaminated development sets. Those choices are data-centric ML choices. They define what the model is asked to learn and what kind of error is considered dangerous.

This distinction matters for future work. It would be easy to interpret the Qwen3.5 numbers as evidence that a larger or newer backbone is the primary solution. The Qwen3.6 comparison argues against that simple reading. A stronger base model helps, but it does not remove the need for a task-specific ontology or a holdout benchmark that measures the right failures. The Gemma comparison also argues against a single-metric reading: a model can be more conservative on one critical-false-positive measure while still less complete on exact match and seen-state recognition. The practical lesson is that model selection should be downstream-risk aware.

The same lesson applies to training data. Adding more examples is useful only when the examples target the remaining decision boundaries. After the current adaptation, page visibility is less interesting than temporally adjacent completion cues, rare positive states, and hard negatives where labels or page names appear without the target page being active. Future data collection should therefore be driven by benchmark failures, not by a desire to enlarge the dataset uniformly.

7.2.3 Implications for ML System Design

The thesis also suggests a broader ML systems pattern. Foundation models are most useful in this setting when the system designer narrows the model's job until its outputs can be tested. The VLM is not asked to understand the whole cockpit; it is asked to predict a small fact vector. The LLM is not asked to run the procedure; it is asked to explain a current evidence packet. The runtime is not asked to trust either model unconditionally; it validates and logs their outputs. This division of labour turns a broad foundation-model capability into a set of smaller contracts.

This pattern differs from simply adding guardrails around a chatbot. Guardrails often operate after a model has produced a free-form answer. In SimTutor, the contract exists before generation: the procedure pack defines the target space, the evidence packet defines what the model is allowed to cite, the visual ontology defines what the VLM may report, and the harness defines what can become an overlay. The model still contributes flexibility, but the shape of the problem is defined outside the model. That is why the thesis can make an auditability claim even though it cannot prove full educational

effectiveness.

For deployable ML systems, this distinction is important. A model-centric approach asks which model is strong enough to solve the task. An evidence- centric approach asks which parts of the task should be assigned to learned models, which parts should remain symbolic or authored, and how conflicts should be resolved. The cockpit tutor benefits from the second approach because the costs of different mistakes are not symmetric. A wrong synonym in an explanation is less serious than a wrong overlay target. A missed visual cue is less serious than a hallucinated completion cue. A schema-invalid VLM response is less dangerous if the runtime rejects it before it becomes evidence.

The resulting system is not simpler than an unconstrained chatbot, but it is more inspectable. Every added constraint creates an artifact: a schema, a source policy, an ontology, a UI map, a benchmark, or a replay fixture. Those artifacts are the engineering cost of turning foundation-model fluency into a research object that can be evaluated.

7.3 Interpretation of Pilot Findings

The formative VR pilot provides descriptive evidence about live use of the system. In the reviewed data, all four with-tutor participants completed the 33-step procedure, while none of the five without-tutor participants completed it. Manual coding also shows zero omission errors in the tutor condition and a mean of 5.6 omission errors in the without-tutor condition. These observations are consistent with the intended function of the tutor: it keeps the current procedure step visible, retrieves relevant procedural context, and can point the learner toward cockpit controls.

The questionnaire data add a second, weaker descriptive layer. After excluding P02, the raw NASA-TLX means were 34.12 without tutor and 23.66 with tutor, and the with-tutor group gave a mean tutor-specific rating of 5.92 on the recorded 1–7 scale. These values are useful for formative interpretation, especially when read together with comments about grounded feedback, missed operations, small VR checklist text, and confusing abbreviations. They should not be read as evidence that the tutor reduced workload or increased trust in a controlled sense.

The same data also show that assistance does not eliminate procedural error. Commission and prompt-assistance errors remained in the tutor condition. This is an important finding because it points to a more precise design problem than "does the tutor help?" The remaining problem is how flexible model-generated guidance, deterministic harness repair, visual-state uncertainty, and human action timing align with a strict procedure

specification. The tutor can reduce missed-step patterns while still leaving room for wrong timing, incomplete execution, or over-trust in assistance.

The live help-cycle logs support the infrastructure claim. Across the four with-tutor trials, the system recorded 77 help cycles, 23 VLM calls, 82 overlay executions, zero overlay rejections, and 21 fallback or repair events. The zero overlay rejection count should not be read as "nothing went wrong." Rather, it means that proposed overlay actions stayed within the configured allowlist after schema validation and harness processing. The repair events are evidence that the constraint layer was active, not evidence that model output was always directly correct.

The pilot therefore has two roles in the thesis. First, it shows that the instrumented tutoring runtime can be used in a live VR setting and produce analysable logs. Second, it reveals the kind of human-system interaction data that a later controlled study should measure more carefully. It does not support causal claims about learning, workload, retention, transfer, or task speed.

7.3.1 Boundary Between Assistance and Collaboration

The pilot also clarifies what is meant by a trustworthy AI collaborator. A collaborator is not simply a system that supplies more instructions. In a high-load VR cockpit, more instructions can increase cognitive work if they arrive at the wrong time, use unfamiliar abbreviations, or force the learner to compare multiple sources of evidence. A trustworthy collaborator must regulate its own certainty, decide when to highlight and when to remain text-only, and help the learner understand the evidence behind the next step.

The with-tutor trials show both sides of this boundary. Completion and omission patterns suggest that step-aware guidance can keep learners oriented. The commission and prompting/assistance errors show that orientation is not enough. The system also has to avoid repeated or stale guidance after a state has already been achieved. P09's comments are especially instructive: the tutor was valued because it identified where the participant was stuck and did not simply advance, but the same participant still reported needing to compare the display with AI instructions and struggling with abbreviations. Trust, in this context, is not blind acceptance. It is an interaction in which the learner can see why the assistant is making a claim and can recover when the claim is incomplete.

This interpretation strengthens the thesis title. To explain reality is not to replace the learner's situational awareness. It is to support it by making the state, the evidence, and the uncertainty easier to act on. The current prototype implements only an early

version of that idea. It has evidence packets, visual facts, validation, and logs, but it does not yet optimise when help should be minimal, when it should be explicit, and when it should ask the learner or an instructor for confirmation.

7.4 Ontology Design Lessons

The ontology evolution is the clearest design lesson in the thesis. The first visual-fact ontology mixed observable facts with procedural conclusions. For example, `ins_go` asked the VLM to decide whether an alignment state had effectively been reached. That target was not simply visual. It depended on particular display text, the absence of other display states, and the procedure phase. The failure pattern in the first benchmark made the boundary problem visible: a single-frame VLM was being asked to perform both perception and procedure reasoning.

The 13-fact ontology corrected that boundary. It decomposed compound targets into lower-level visual observations, such as whether the ground-alignment text or OK text is visible, and decomposed FCS-MC states into visually distinguishable stages. The downstream runtime then interprets those observations together with telemetry and procedural context. The design principle is therefore:

> **Visual fact targets should be low-level, locally verifiable visual propositions. Higher-level procedural state should be inferred downstream from visual facts, telemetry, temporal context, and procedure rules.**

This principle matters beyond label naming. It affects annotation quality, training signal quality, error diagnosis, and runtime safety. A human reviewer can judge whether a piece of text is visible in a screenshot without knowing the procedure history. The same reviewer cannot always judge whether a procedure state is complete without knowing what happened before the image. When the fact contract respects that distinction, the model's output becomes easier to review and safer to combine with deterministic gates.

Residual false positives on completion-type facts show the remaining edge of the method. Some states are visually similar across intermediate and final phases, especially when numeric readouts or transient display states are involved. Those cases may require more balanced data, higher-resolution crops, repeated observations, or explicitly temporal VLM inputs. The thesis therefore does not claim that the 13-fact ontology solves visual grounding generally. It offers a tested design discipline for one demanding

procedure and a concrete method for discovering when a visual target is still too close to a procedural conclusion.

As a heuristic for future domains, the rule is simple: if a fact cannot be labelled from one image without knowing the procedure step, it is probably too high-level for the VLM contract. It should be decomposed into visible atoms or deferred to the procedure engine. This is the point at which the thesis moves from a case-specific implementation detail to a reusable representation lesson.

7.5 Limitations

The contribution should be read inside the following limits.

7.5.1 Evaluation Scope

All datasets, replay cases, and pilot trials concern one procedure: the F/A-18C cold-start. The visual extractor is trained and evaluated on a fixed composite cockpit panel with the same aircraft type and rendering environment. The holdout sets are independent sessions with zero training overlap, but they still measure within-domain generalisation. They do not establish cross-procedure, cross-aircraft, or cross-simulator generality.

The largest training recipe contains 928 bilingual SFT rows after the Run-005 oversampling step. This is deliberately small and practically useful for a domain-specific adaptation study, but it does not identify the minimum data requirement, the performance ceiling, or the effect of each recipe component. The thesis does not include ablations for bilingual rows, oversampling, LoRA rank, composition rebalance, or alternative fine-tuning methods.

7.5.2 System and VLM Scope

The VLM is evaluated as a single-frame visual-fact extractor. It does not decide procedure progress, select final tutor actions, or control the overlay directly. Residual critical false positives on completion-like facts remain important because a false visual observation can still influence downstream reasoning if it is not checked by telemetry, temporal confirmation, or harness constraints. Sticky facts and confirmation policies mitigate this risk, but their complete end-to-end effect has not been isolated experimentally.

The backbone comparison covers Qwen3.5-9B, Qwen3.6-27B, and Gemma-4-31B under a matched LoRA-style adaptation recipe. This is useful model-diversity evidence

within the scope of the project, but it does not cover other VLM families, larger models, full fine-tuning, alternative adapters, or non-cockpit visual domains.

The runtime is implemented and exercised through replay, fixtures, desktop DCS World, and the Quest 3 pilot. However, headset latency, overlay legibility, gaze or hand-tracking accuracy, and expert-rated response usefulness were not measured as controlled system-level outcomes.

7.5.3 Construct Validity

Several constructs in the thesis are proxies. The VLM benchmark measures single-frame visual fact extraction, not human understanding of cockpit displays. Replay fixtures measure whether known evidence-grounding cases are handled, not whether every possible live help request is safe. Manual step completion measures whether procedure states were reached, not whether the learner can later perform the task unaided. Raw NASA-TLX values measure reported workload after one formative trial, not stable workload reduction.

These proxies are still useful because they match the development stage of the system. Before testing learning outcomes, the tutor must be able to observe state, produce bounded visual facts, validate model output, and export interpretable logs. The constructs chosen here therefore prioritise technical readiness and descriptive interaction evidence. A later controlled study should add constructs that the current thesis does not measure: retention after a delay, transfer to a different start-up scenario, expert-rated explanation quality, overlay legibility in headset, and calibrated trust.

The construct-validity boundary also protects the ML results from overreach. High VLM accuracy is necessary for the visual evidence channel, but it is not a direct measure of explanation quality. Conversely, a participant may rate the tutor positively even if some visual facts or overlays required repair. The evaluation therefore keeps component reliability, system auditability, and human response as separate constructs.

7.5.4 Pilot Scope

The pilot contains nine participants: four with tutor and five without tutor. Assignment was not randomised, prior DCS World experience was not balanced, and the sample is too small for inferential statistics. Without-tutor trials stopped at different procedure points, so their durations are not comparable to completed with-tutor trials.

Questionnaire evidence is usable but limited. P02's questionnaire metadata could not

be confidently attributed and is excluded from all questionnaire- derived summaries. The remaining raw NASA-TLX, tutor-rating, usability/user rating, and free-text comment data are descriptive only. They support formative interpretation, not workload, confidence, trust, or usability claims with inferential strength.

Finally, manual review inferred without-tutor completion from passive telemetry gates and the action timeline. Because the active VLM pathway was not available in that condition, visually evidenced steps are less certain. This limitation does not invalidate the formative finding, but it shows why the next study needs stricter instrumentation and coding.

7.5.5 Scope Summary

Within these limits, the thesis supports four claims: an evidence-constrained procedural tutoring runtime can be implemented with explicit architecture boundaries; a small-data VLM adaptation pipeline can produce reliable visual-fact extraction on local holdouts; the visual ontology evolution provides a concrete design lesson about perception and reasoning; and the VR pilot shows that the system can be exercised with participants while producing analysable logs. It does not establish controlled educational effectiveness.

7.6 Future Work

The limitations above lead to one larger research agenda: how can AI become a trustworthy collaborator in dynamic, high-load, multimodal training environments? In this thesis, trustworthiness does not mean that the model sounds confident. It means that the assistant can expose the evidence behind its guidance, know when a modality is unavailable, defer to explicit procedure rules, recover from uncertainty, and leave an audit trail that instructors can inspect. The following directions extend that agenda.

7.6.1 Controlled Human-Subject Evaluation

The next study should compare the grounded tutor against a baseline condition with randomised assignment, balanced participant metadata, pre-defined coding rules, and complete questionnaire and workload capture. A two-condition design would be more realistic than the original three-condition proposal for the next iteration, because the

current priority is to establish whether the completed runtime produces measurable differences in completion, errors, workload, and retention under controlled conditions.

7.6.2 Ontology Extension to Other Procedures

The 13-fact ontology should be extended to additional procedures only after a new task analysis identifies which states are telemetry-visible, vision-priority, or outside the current evidence-source contract. Each new procedure needs its own visual fact contract, reviewed data, holdout benchmark, and runtime bindings. This is the appropriate test of whether the ontology discipline transfers beyond cold-start.

7.6.3 Temporal and Multi-Frame Visual Reasoning

Several residual errors involve states that are difficult to disambiguate from a single screenshot. Multi-frame inputs, short visual histories, or repeated confirmation gates could help distinguish intermediate and final display states. This would require changes to capture, annotation, model input format, and benchmark metrics.

7.6.4 System-Level Help-Cycle Evaluation

A separate evaluation should measure the full help cycle rather than individual components: trigger-to-overlay latency, correctness of highlighted targets, frequency and type of harness repair, response usefulness under expert review, and whether vision-priority steps improve over telemetry-only baselines. This would connect the RQ1 and RQ2 technical evidence more directly to RQ3.

For the trustworthy-collaborator agenda, this system-level evaluation should also measure when the assistant should stay silent, ask for confirmation, or state uncertainty instead of giving another instruction. In high-load VR use, too much guidance can be as harmful as too little guidance if it competes with the learner's attention or obscures the evidence needed for action.

7.6.5 Multimodal Collaboration Policies

Future work should treat modality selection as a policy problem. The assistant can respond with text, a spatial overlay, a request for visual confirmation, a retrieved manual excerpt, or silence. Each modality has a cost. Text consumes reading and memory resources. Overlays consume visual attention and may pull the learner toward one

control. VLM calls consume latency and may return uncertain facts. Retrieval can add authority but can also lengthen the message. A trustworthy collaborator should choose the lightest modality that is sufficient for the current evidence state.

Such a policy could begin deterministically. For example, if telemetry already confirms completion, the tutor should avoid repeating an instruction. If a visual fact is uncertain, it should ask for confirmation rather than highlight a completion target. If the learner is on a known high-confusion step, the tutor could give a shorter explanation first and reveal more detail only on request. Later work could learn parts of this policy from expert corrections, but the policy should remain auditable: every modality choice should still be tied to state, evidence, and uncertainty.

7.6.6 In-Situ Correction and Adaptation

The current VLM adaptation pipeline is offline. Future versions could collect instructor or learner corrections when a visual fact is wrong and feed those corrections into later review or incremental adaptation. This direction requires careful safeguards because in-situ labels may be noisy, and adaptation must not erase previously reliable behaviour.

7.6.7 Summary of Future Directions

Table 7.1 summarises the main future-work directions and the limitation each direction addresses.

Table 7.1: Summary of future work directions.

Direction	**Current status**	**Main challenge**
Controlled human-subject study	Formative pilot completed; causal evidence pending	Randomisation, metadata, workload, and statistical power
Ontology extension	Cold-start ontology evaluated only within current cockpit layout	New task analysis, annotation, and holdout design
Temporal visual reasoning	Current VLM uses single composite-panel frames	Multi-frame data, model capacity, and temporal metrics
System-level help-cycle evaluation	Component and replay evidence available	Defining integrated latency, quality, and repair metrics
In-situ correction and adaptation	Offline review and LoRA pipeline implemented	Noisy labels, forgetting, and safe update policies

These directions keep the thesis spine intact. The next stage is not to make the foundation model less constrained, but to strengthen the evidence loop around it: broader

procedure packs, better visual facts, richer temporal evidence, controlled human evaluation, and logs that make every tutor response auditable after the fact.

8 Conclusion

This thesis began from a practical problem shared by many procedural training settings: a learner often needs help while acting inside a complex simulated system, but useful help must be grounded in what is currently true. A fluent foundation-model response is not enough when the next correct action depends on telemetry, procedure order, display state, and earlier actions. The central answer developed in this thesis is that foundation models can be made suitable for procedural tutoring when they are placed inside an auditable evidence loop.

The F/A-18C cold-start task provided a demanding case for that idea. It required the tutor to combine telemetry-visible cockpit state, visual evidence from display pages, retrieved procedural knowledge, unavailable-source boundaries, and validation before producing overlay or textual guidance. The result is not a claim that learning effectiveness has already been proven. It is a technical and methodological contribution: a way to make model-assisted procedural help state-aware, inspectable, and bounded by available evidence.

8.1 Summary of Completed Work

The completed work answers the three research questions as follows.

RQ1: evidence integration and tutoring architecture. The thesis shows how simulator telemetry, retrieved procedural sources, and visual fact observations can be integrated into a constrained help cycle. The runtime builds an evidence packet before model generation, infers candidate procedure steps from declarative gates, asks the model to respond inside a pack-derived schema, validates or repairs the response through a harness, and executes only allowlisted overlay actions. Logging and replay then make the resulting help cycle inspectable after the fact. This answers RQ1 at the level of technical auditability: the LLM is not the authority for procedure state, but a generator constrained by evidence, schema, harness, and procedure-pack rules.

RQ2: visual fact extraction and ontology design. The thesis also shows that small-data VLM adaptation can support reliable visual-fact extraction for the cockpit display states required by the tutor. The 13-fact ontology and LoRA training pipeline produce the strongest primary result on the Qwen3.5-9B line, with approximately 0.99 fact accuracy on two independent 13-fact holdouts and sharply reduced critical false positives relative to the base model. Supplementary Qwen3.6-27B and Gemma-4-31B results support the same data-and-ontology recipe within the evaluated backbones. The more general lesson is representational: the VLM should report locally verifiable visual facts, while procedural completion should be inferred downstream from visual facts, telemetry, time, and procedure context.

RQ3: formative VR pilot evidence. The completed Quest 3 pilot provides formative evidence about whether the runtime and instrumentation can be exercised with participants in immersive simulation. In the reviewed data, the four with-tutor participants completed the 33-step procedure, while the five without-tutor participants did not. Omission errors disappeared in the tutor condition, while assistance-coupled errors remained. The help-cycle logs also show that the tutor can operate live with VLM calls, overlay actions, harness repairs, and replayable exports. These findings answer RQ3 descriptively. They support feasibility, instrumentation readiness, and error-pattern analysis; they do not establish causal learning gains, workload reduction, retention, transfer, or speed improvement.

Together, these answers define the thesis contribution. First, the work provides an evidence-grounded tutoring runtime pattern for procedural simulation, where generated help is downstream of explicit state evidence and validation. Second, it contributes a visual-fact ontology and adaptation method that separates VLM perception from procedural reasoning. Third, it delivers a pilot-ready evaluation path for studying grounded tutoring with real VR participants. The important point across all three is not the presence of an LLM or VLM by itself, but the boundary placed around foundation-model output.

That boundary is the main transferable idea. A foundation model can be useful in a high-fidelity procedural trainer, but only when its role is made explicit. The LLM is a language and explanation component, not the source of simulator truth. The VLM is a perception component, not the procedure engine. Retrieval is a source of textual grounding, not a licence to mix procedure variants. Telemetry is authoritative where it exists, but it is not a complete pedagogy. The runtime contribution is the mech-

anism that combines these partial authorities into one help cycle and records how the combination was made.

The ML contribution is similarly bounded but concrete. The thesis does not claim that LoRA adaptation solves cockpit perception in general. It shows that with a carefully revised ontology, reviewed small data, composition rebalance, and independent holdouts, a VLM can become substantially more reliable on the specific structured visual facts needed by the tutor. The Qwen3.5, Qwen3.6, and Gemma comparison shows why this should be treated as a model-and-data decision rather than a simple choice of the largest backbone. Reliability depends on the fact contract, the error cost, and the downstream use of the prediction.

The scope remains deliberately bounded. The evaluation covers one procedure, one cockpit layout, local visual holdouts, replay and harness readiness, and a small non-random pilot. The current evidence supports technical reliability and auditability more strongly than educational effectiveness. It does not show that the tutor generalises across domains or that the pilot differences were caused by the tutor alone. Those boundaries are not weaknesses to hide; they are what make the contribution inspectable.

8.2 Future Work

The immediate next step is a controlled human subject evaluation. Such a study should keep the current export and replay infrastructure, but add randomised assignment, balanced participant metadata, complete questionnaire and workload capture, pre-defined coding rules, and a sample large enough for inferential analysis. Only that kind of study can test whether, and how, the technically grounded help cycle changes learning, workload, retention, or transfer.

The technical agenda is equally clear. The procedure-pack and visual-fact approach should be extended to additional procedures, but each extension needs new task analysis, evidence-source mapping, visual ontology design, reviewed data, and independent holdout evaluation. The VLM pipeline should also move beyond isolated screenshots where procedure state depends on temporal evidence: short visual histories, repeated confirmation, or multi-frame inputs are natural next mechanisms for completion-like facts. Finally, the full help cycle should be evaluated as an integrated system, including latency, overlay correctness, harness repair frequency, expert-rated response usefulness, and the moments where the assistant should withhold, defer, or explicitly state uncertainty.

A longer-term version of the system could learn from in-situ corrections by instructors or learners. That direction would turn live tutoring into a data collection loop, but it requires careful safeguards: correction labels may be noisy, updates must not erase already reliable behaviour, and every adapted model must still be evaluated against untouched holdouts.

The broader research question is how to make AI a collaborator rather than an answer generator. In the setting studied here, collaboration means helping the learner act under uncertainty while preserving the learner's ability to inspect the situation. It means choosing when to highlight, when to explain, when to ask for confirmation, and when to remain silent. It also means leaving enough evidence behind that an instructor, researcher, or developer can reconstruct what the system believed and why it acted. The prototype in this thesis does not complete that agenda, but it establishes the evidence loop needed to pursue it.

The title of this thesis, *Explain Reality*, captures the final design commitment. A procedural tutor should not merely explain a checklist, and it should not merely generate plausible language about a task. It should explain the learner's current reality: what is observable, which evidence supports it, which procedure rule applies, what remains uncertain, and why the next response is allowed. That is also the long-term path toward AI as a trustworthy collaborator in dynamic, high-load, multimodal environments. The contribution of this thesis is an architecture and evidence method for making that explanation auditable.

References

[1] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, et al. Do as i can, not as i say: Grounding language in robotic affordances. In *Proceedings of the 6th Conference on Robot Learning*, 2022. doi: 10.48550/arXiv.2204.01691.

[2] Vincent Aleven, Bruce M. McLaren, Jonathan Sewall, and Kenneth R. Koedinger. A new paradigm for intelligent tutoring systems: Example-tracing tutors. *International Journal of Artificial Intelligence in Education*, 19(2):105–154, 2009. doi: 10.3233/IRG-2009-19(2)02.

[3] Saleema Amershi, Dan Weld, Mihaela Vorvoreanu, Adam Fourney, Besmira Nushi, Penny Collisson, Jina Suh, Shamsi Iqbal, Paul N. Bennett, Kori Inkpen, Jaime Teevan, Ruth Kikin-Gil, and Eric Horvitz. Guidelines for human-AI interaction. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, pages 1–13, 2019. doi: 10.1145/3290605.3300233.

[4] Jinze Bai, Shuai Bai, Shusheng Yang, Shijie Wang, Sinan Tan, Peng Wang, Junyang Lin, Chang Zhou, and Jingren Zhou. Qwen-VL: A versatile vision-language model for understanding, localization, text reading, and beyond. *arXiv preprint arXiv:2308.12966*, 2023.

[5] Wenliang Dai, Junnan Li, Dongxu Li, Anthony Meng Huat Tiong, Junqi Zhao, Weisheng Wang, Boyang Li, Pascale Fung, and Steven Hoi. InstructBLIP: Towards general-purpose vision-language models with instruction tuning. In *Advances in Neural Information Processing Systems*, volume 36, 2023. doi: 10.48550/arXiv.2305.06500.

[6] DCS-BIOS Project. DCS-BIOS documentation, n.d. DCS-BIOS v0.10.0 documentation. Accessed 2026-06-05. Available at: https://dcs-bios.readthedocs.io/en/latest/.

[7] Eagle Dynamics. DCS World, 2013. Digital Combat Simulator. Available at: https://www.digitalcombatsimulator.com/.

[8] Eagle Dynamics. DCS: F/A-18C early access guide, 2023. Updated 18 July 2023. Available at: https://www.digitalcombatsimulator.com/en/downloads/documentation/dcs-hornet_early_access_guide_en/.

[9] David Gunning, Mark Stefik, Jaesik Choi, Timothy Miller, Simone Stumpf, and Guang-Zhong Yang. XAI—explainable artificial intelligence. *Science Robotics*, 4 (37):eaay7120, 2019. doi: 10.1126/scirobotics.aay7120.

[10] Daniel Han and Michael Han. Unsloth: Open-source library for faster LLM fine-tuning, 2024. URL https://github.com/unslothai/unsloth.

[11] Sandra G. Hart and Lowell E. Staveland. Development of NASA-TLX (Task Load Index): Results of empirical and theoretical research. In Peter A. Hancock and Najmedin Meshkati, editors, *Human Mental Workload*, volume 52 of *Advances in Psychology*, pages 139–183. North-Holland, 1988. doi: 10.1016/S0166-4115(08)62386-9.

[12] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

[13] Enkelejda Kasneci, Kathrin Seßler, Stefan Küchemann, Maria Bannert, Daryna Dementieva, Frank Fischer, Urs Gasser, Georg Groh, Stephan Günnemann, Eyke Hüllermeier, Stephan Krusche, Gitta Kutyniok, Tilman Michaeli, Claudia Nerdel, Jürgen Pfeffer, Oleksandra Poquet, Michael Sailer, Albrecht Schmidt, Tina Seidel, Matthias Stadler, and Gjergji Kasneci. ChatGPT for good? on opportunities and challenges of large language models for education. *Learning and Individual Differences*, 103:102274, 2023. doi: 10.1016/j.lindif.2023.102274.

[14] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474, 2020.

[15] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. *Advances in Neural Information Processing Systems*, 36, 2023. doi: 10.48550/arXiv.2304.08485.

[16] Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, Sayak Paul, and Benjamin Bossan. PEFT: State-of-the-art parameter-efficient fine-tuning methods, 2022. URL https://github.com/huggingface/peft.

[17] Antonija Mitrovic. Fifteen years of constraint-based tutors: What we have achieved and where we are going. *User Modeling and User-Adapted Interaction*, 22(1–2):39–72, 2012. doi: 10.1007/s11257-011-9105-9.

[18] Riccardo Palmarini, John Ahmet Erkoyuncu, Rajkumar Roy, and Hosein Torabmostaedi. A systematic review of augmented reality applications in maintenance. *Robotics and Computer-Integrated Manufacturing*, 49:215–228, 2018. doi: 10.1016/j.rcim.2017.06.002.

[19] Qwen Team. Qwen3-VL technical report, 2025. URL https://arxiv.org/abs/2511.21631.

[20] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009. doi: 10.1561/1500000019.

[21] Maxim Tkachenko, Mikhail Malyuk, Andrey Holmanyuk, and Nikolai Liubimov. Label Studio: Data labeling software, 2020. URL https://github.com/HumanSignal/label-studio.

[22] Kurt VanLehn. The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems. *Educational Psychologist*, 46(4):197–221, 2011. doi: 10.1080/00461520.2011.611369.

[23] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[24] Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, and Shengyi Huang. TRL: Transformer reinforcement learning, 2020. URL https://github.com/huggingface/trl.

[25] Bryan Wang, Gang Li, Xin Zhou, Zhourong Chen, Tovi Grossman, and Yang Li. Screen2Words: Automatic mobile UI summarization with multimodal learning. *arXiv preprint arXiv:2108.03353*, 2021. doi: 10.48550/arXiv.2108.03353.

[26] Yu Zhang. Explain reality: Grounded procedural tutoring with foundation models and visual fact extraction in immersive simulation — MSc thesis proposal. TU Clausthal, Institute for Software and Systems Engineering (ISSE). Unpublished manuscript., 2025.

9 Appendix

This appendix supports the methodology, implementation, and evaluation chapters. It preserves raw configuration, dataset, replay, diagram, and pilot-coding material without extending the claim boundaries stated in the main chapters.

9.1 Full LoRA Hyperparameter Configuration

Table 9.1 lists the complete hyperparameter configuration for all three backbone models under the matched Run-003 + Run-005×2 recipe. Values are drawn from available training reports and benchmark artifacts.

Note: training loss values are not directly comparable across backbones due to differences in tokenizer behaviour, chat-template structure, target-sequence distribution, and model parameterisation.

9.2 8-Fact V1 Baseline Benchmark

The earliest complete benchmark uses an 8-fact LoRA adapter trained on Run-001 (180 tasks) and evaluated on the Run-002 holdout (50 samples). This line predates the 13-fact ontology and is included for historical reference.

The LoRA adapter improves fact-level metrics substantially but increases critical false positives from 13 to 15. This historical result illustrates the 8-fact v1 ontology's structural weakness: compound targets such as `ins_go` were not visually verifiable from single frames and produced systematic false-positive errors on completion-type facts.

9.3 Dataset Fact Distributions

Table 9.3 reports per-fact `seen` counts for the two main training sources (Run-003, Run-005) and the two 13-fact holdout sets (Run-002 newfacts, Run-004 random). Counts are drawn from the corresponding `stats.json` files. The table uses short display labels;

Table 9.1: Complete LoRA fine-tuning hyperparameters across three backbones.

Parameter	**Qwen3.5-9B**	**Qwen3.6-27B**	**Gemma-4-31B**
`max_seq_length`	4096	4096	4096
`num_train_epochs`	4.0	4.0	4.0
`learning_rate`	2×10^{-4}	2×10^{-4}	2×10^{-4}
`per_device_train_batch_size`	1	1	1
`gradient_accumulation_steps`	4	4	4
`lora_r`	16	16	16
`lora_alpha`	16	16	16
`lora_dropout`	0.0	0.0	0.0
`seed`	3407	3407	3407
`load_in_4bit`	True	True	True
`warmup_ratio`	0.1	0.1	0.1
`weight_decay`	0.01	0.01	0.01
`train_rows`	928	928	928
`eval_rows`	0	0	0
Backbone-specific			
Vision LoRA	enabled	enabled	disabled
LoRA target modules	vision+lang	vision+lang	all-linear
Chat template	Qwen default	Qwen default	gemma-4
GPU memory util.	0.6	0.6	0.95
Training outcomes			
Train runtime (s)	6419	12619	8050
Train loss (final)	0.2156	0.1068	0.0474

Table 9.2: 8-fact v1 benchmark: Qwen3.5-9B Base vs. LoRA-v1 on Run-002 holdout (50 samples, EN).

Metric	**Base**	**LoRA-v1**	**Δ**
JSON valid rate	1.0	1.0	—
Schema valid rate	1.0	1.0	—
Fact accuracy	0.7600	0.9150	+0.1550
Macro F_1	0.4241	0.5847	+0.1605
Seen F_1	0.4714	0.8380	+0.3666
Sample exact match	0.06	0.36	+0.30
Critical false positives	13	15	+2

the authoritative fact identifiers are defined in the visual-fact ontology file used by the procedure pack.

Table 9.3: Per-fact `seen` counts across training and holdout datasets (13-fact v2 ontology).

Fact label	Run-003	Run-005	Run-002	Run-004
TAC page visible	46	24	23	3
SUPT page visible	20	34	5	3
FCS page visible	30	40	13	94
FCS X marks visible	16	18	1	5
BIT root page visible	18	43	9	16
FCS-MC page visible	55	56	9	84
FCS-MC intermediate result	18	15	1	7
FCS-MC in-test state	18	19	6	17
FCS-MC final GO result	18	22	2	60
HSI page visible	106	109	49	100
HSI map layer visible	79	37	27	50
INS GRND alignment text	58	72	42	69
INS OK text	12	14	4	0

Run-005 increases coverage for several sparse targets when combined with Run-003: SUPT page observations rise from 20 to 54, FCS-MC page observations from 55 to 111, and INS GRND alignment text observations from 58 to 130. This table documents data provenance; Chapter 6 evaluates whether the resulting training recipe improves model behaviour on holdout sets.

9.4 CLI Command Reference

The `simtutor` package provides the following CLI commands:

`run`. Executes a simulation using a mock environment adapter with a pre-recorded scenario file and writes the event log.

`replay`. Validates an existing event log against a procedure pack, checking step ordering and state-machine consistency.

`score`. Scores an event log using the scoring engine, producing omission counts, state-violation counts, and a total error score.

`record-vlm`. Runs VLM fact extraction on a set of vision observations and records predictions for benchmarking.

`replay-bios`. Replays a raw DCS-BIOS capture through the telemetry pipeline and optionally invokes the help generation loop.

`validate`. Validates JSONL files against a named schema from the schema registry.

All commands accept model configuration overrides (`-model-provider`, `-model-name`, `-model-base-url`, `-model-timeout-s`) and a language switch (`-lang`).

9.5 Model Provider Configuration

Model access is configured through environment variables and validated at runtime:

Provider. `SIMTUTOR_MODEL_PROVIDER` selects `openai_compat`, `ollama`, or `stub`.

Base URL. `SIMTUTOR_MODEL_BASE_URL` sets the provider endpoint.

Model. `SIMTUTOR_MODEL_NAME` selects the model name.

Timeout. `SIMTUTOR_MODEL_TIMEOUT_S` sets the request timeout.

Credentials. `SIMTUTOR_MODEL_API_KEY` is required for the `openai_compat` provider. API keys are redacted from log output.

Images. `SIMTUTOR_MODEL_ENABLE_MULTIMODAL` toggles base64 image encoding in requests.

Language. `SIMTUTOR_LANG` selects `zh` or `en`.

9.6 Replay Evaluation Suite

Table 9.4 lists the five replay evaluation cases in the `fa18c_startup_v04` suite. Display labels are shortened here; the suite file preserves the exact `case_id` strings.

Cases marked as vision available include per-frame sidecar files (`frames.jsonl`) in the corresponding DCS World Saved Games capture. These files allow vision-dependent replay cases to run without a live DCS World instance.

Table 9.4: Replay evaluation cases.

Case	Step	Vision	Target
No-op capture	S01	no	battery switch
Battery on	S02	no	fire-test switch
Battery on, engine generator off	S01	no	left generator switch
FCS reset / FCS BIT	S18	yes	FCS BIT switch
INS mode	S12	yes	INS mode knob

9.7 Supplementary Architecture and Export Diagrams

The main chapters use simplified diagrams to support the research argument. Figures 9.1, 9.2, and 9.3 retain additional engineering detail for reproducibility and operational context.

9.8 VR Pilot Study — Supplementary Data

Table 9.5 lists participant experience profiles used for descriptive interpretation. P02's questionnaire-derived fields are excluded from workload, confidence, usability, trust, and comment summaries because the available questionnaire metadata contains an internal participant-id mismatch. P08 uses manual completion coding, and P09 is treated as an independent participant after the correction recorded in the trial metadata.

Table 9.5: Pilot participant experience profiles (self-reported).

P	Condition	Group	DCS World	F/A-18C	Cold start	VR
P01	No tutor	novice	none	none	none	very little
P02	No tutor	intermediate	frequent	experienced	tried before	frequent
P03	No tutor	intermediate	occasional	tried before	tried before	occasional
P04	Tutor	intermediate	occasional	tried before	tried before	occasional
P05	Tutor	expert	frequent	experienced	can complete	frequent
P06	No tutor	expert	frequent	experienced	can complete	frequent
P07	No tutor	intermediate	occasional	tried before	tried before	occasional
P08	Tutor	intermediate	occasional	tried before	tried before	occasional
P09	Tutor	novice	none	none	none	very little

Table 9.6 lists the specific steps not completed by each without-tutor participant.

Table 9.7 reports the auto-coded error counts (before manual review) alongside the manual error counts, showing how much the automated export had to be corrected before pilot interpretation.

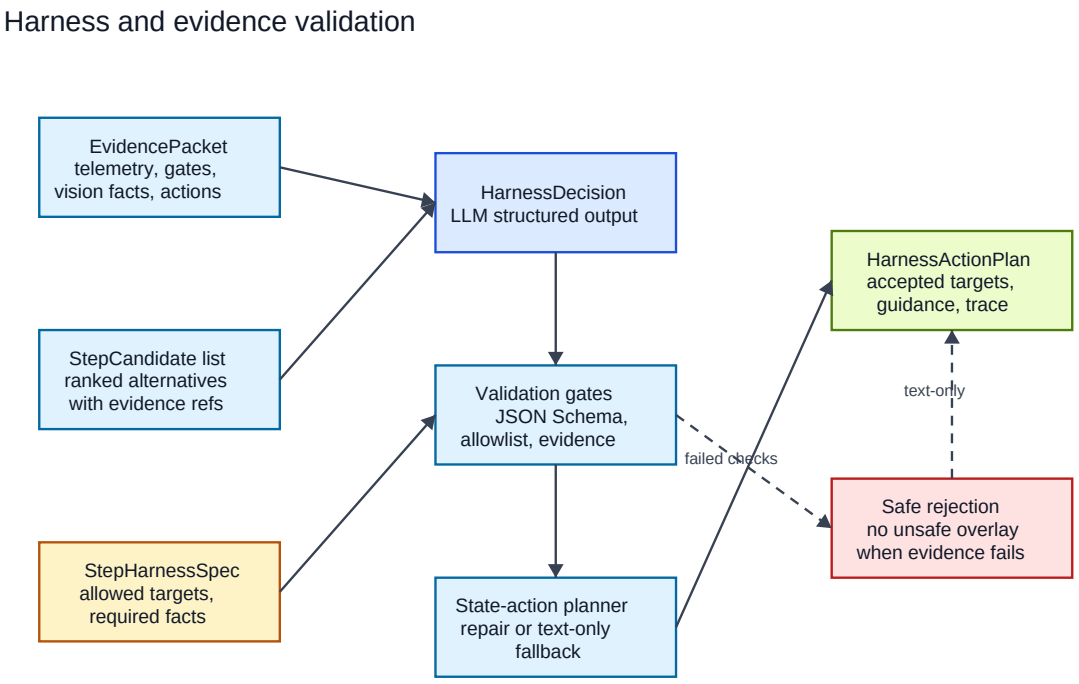


Figure 9.1: Harness and evidence validation. Candidate tutor actions are checked against schema constraints, UI allowlists, evidence consistency, and procedure state before an overlay action plan is accepted; unsafe or under-grounded actions are repaired, rejected, or downgraded to text-only guidance.

Table 9.6: Steps not completed in the without-tutor condition (manual coding).

P	Steps not completed
P01	S09, S12, S18, S19, S20, S22, S24, S27, S29, S31
P02	S02, S14, S18, S19, S27, S30
P03	S07, S09, S16, S18, S19, S26, S27, S29, S32
P06	S07, S18, S19, S27
P07	S02, S09, S12, S13, S16, S17, S18, S19, S22, S26, S27, S28, S29, S32

S18 (FCS-MC BIT page on right DDI) and S19 (FCS BIT final GO result) are missed by all five without-tutor participants and are vision-priority display checks. S27 (flap switch to AUTO) is also missed by all five, but the procedure pack classifies it as BIOS-priority with variable and gate evidence rather than VLM-bound visual evidence. The pattern is therefore descriptive: it shows where participants struggled, not that one evidence source explains the omissions.

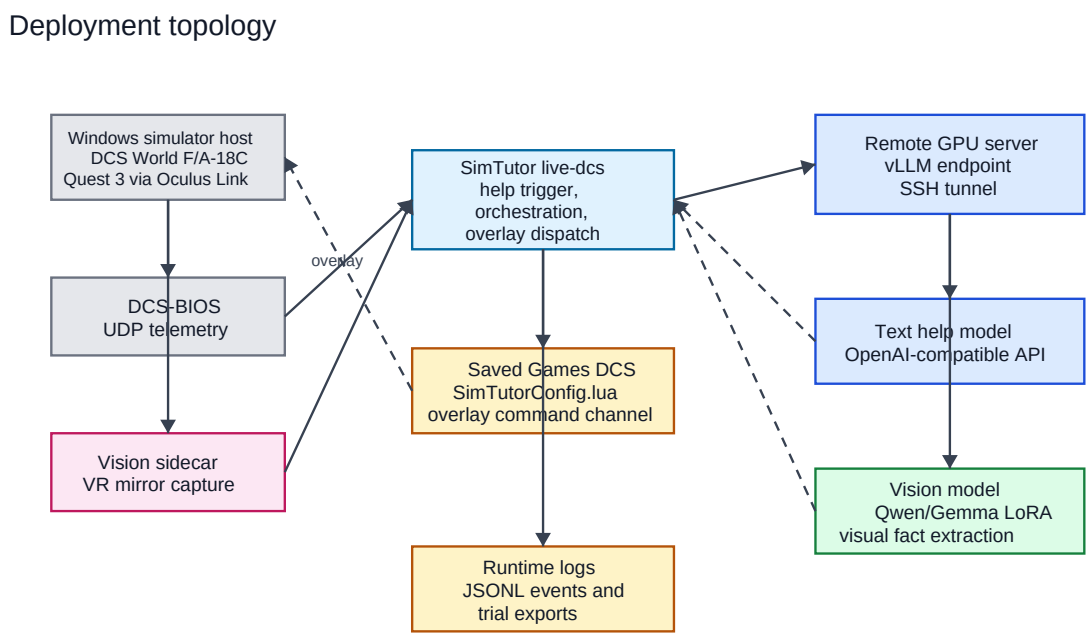


Figure 9.2: Deployment topology used by the prototype. The simulator, Quest 3 overlay path, local runtime, replay utilities, and remote model providers are separated so that live trials and offline analysis can use the same evidence and logging contracts.

Table 9.7: Auto-coded vs. manual error counts per participant.

P	Cond.	Auto OM	Auto CO	Man. OM	Man. CO	Man. total
P01	No tutor	4	3	8	0	10
P02	No tutor	0	3	0	0	1
P03	No tutor	0	4	7	0	11
P06	No tutor	0	3	2	0	4
P07	No tutor	1	9	11	0	11
P04	Tutor	0	4	0	3	7
P05	Tutor	0	3	0	3	6
P08	Tutor	0	3	0	0	0
P09	Tutor	0	3	0	3	5

Auto-coded errors are from the automated pipeline (Auto_Error_* columns in `step_coding.csv`). Manual errors are from the human-reviewed columns (Error_*). P08's auto errors were all overridden to zero by manual review (see data-quality note in Section 6.3).

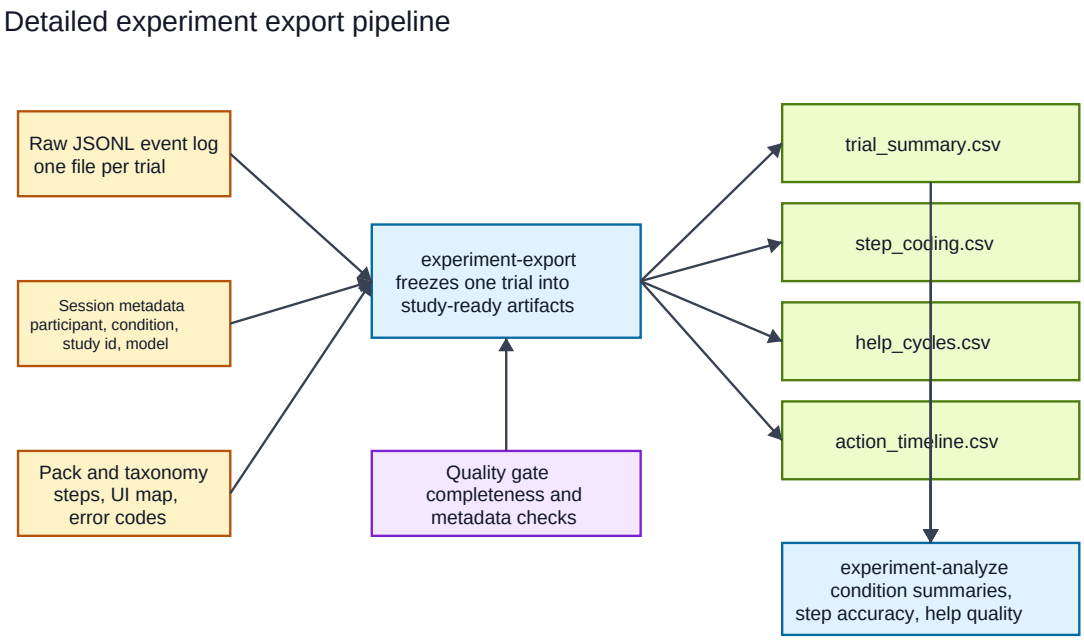


Figure 9.3: Detailed experiment export pipeline. Runtime event logs and trial metadata are transformed into participant-level summaries, step coding, help-cycle records, quality gates, and study-level analysis tables.